70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A3

Sun'iy intellekt tizimlarini optimallashtirish va unumdorligini baholash

Bajarish bo‘yicha qo‘llanma: 6 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A3
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A3
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu virtual laboratoriyada siz o'qitilgan neyron tarmoqni ishlab chiqarishga tayyorlash zanjirini to'liq bajarasiz: PyTorch modelini ONNX'ga eksport qilish, ONNX Runtime'da grafik optimallashtirish darajalari va paket o'lchamini o'lchash, PyTorch va ONNX Runtime'da INT8 kvantlash (dinamik va statik, turli kalibrlash usullari), so'ngra aniqlik-tezlik-hajm jadvali va Pareto tahlili. Real labdan farqi: ResNet-50 va ImageNet o'rniga sklearn digits (8x8) va kichik CNN/MLP, GPU/TensorRT/OpenVINO yo'q — ular o'rniga CPU muqobillari va «GPU'da bu qadam qanday bo'ladi» izohlari; sandbox CPU'da AVX/VNNI yo'qligi sababli INT8 tezlanishi kutilmaydi (bu o'zi muhim saboq).

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A3`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A3 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Etalon: PyTorch FP32 o'qitish va ONNX eksport
2. 2-qadam. ONNX Runtime: optimallashtirish darajalari, paket, oqimlar, Little qonuni
3. 3-qadam. PyTorch'da INT8 dinamik kvantlash (Linear)
4. 4-qadam. ONNX Runtime'da kvantlash: dinamik va statik (MinMax, Percentile, shovqin)
5. 5-qadam. Yakuniy jadval, Pareto tahlili va grafik
6. 6-qadam. Energiya bahosi, nusxalar rejasi va GPU/TensorRT yo'l xaritasi
7. Qo‘shimcha mashqlar (10)
8. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Etalon: PyTorch FP32 o'qitish va ONNX eksport

MAQSAD

Kichik CNN sklearn digits to'plamida 3 epoxa o'qitiladi, batch 1 kechikishi (p50/p95) o'lchanadi, so'ngra model `torch.onnx.export` bilan dinamik paket o'qi va opset 17 bilan eksport qilinadi. `onnx.checker` grafikni tekshiradi, ONNX Runtime chiqishi PyTorch bilan `np.allclose` orqali solishtiriladi. O'zgartirib ko'ring: epoxa sonini, `dynamic_axes` ni olib tashlab kirish shaklini, `do_constant_folding=False` da tugunlar sonini.

> **Kutilgan natija:** 3 epoxadan keyin ~94 % aniqlik, PyTorch eager p50 ~0,15 ms; ONNX grafikda 10 tugun (BatchNorm Conv'ga birlashgan), maksimal farq ~1e-6, «MOS».

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Etalon: kichik CNN'ni PyTorch'da o'qitish, ONNX'ga eksport qilish va moslikni
tekshirish
import warnings; warnings.filterwarnings("ignore")
import os, time, numpy as np, torch, torch.nn as nn, onnx, onnxruntime as ort
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); np.random.seed(0)
torch.set_num_threads(2)                  # sandbox'da 2 yadro bor
torch.backends.nnpack.set_flags(False)    # eski CPU'da NNPACK ogohlantirishini o'chiramiz

# --- Ma'lumot: sklearn digits — 1797 ta 8x8 kulrang rasm, 10 sinf (ImageNet o'rniga, CPU
uchun)
X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype(np.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Xtr, Xte, ytr, yte = map(torch.tensor, (Xtr, Xte, ytr, yte))

# --- Model: ResNet-50 o'rniga kichik CNN (Conv-BN-ReLU-Pool x2 + 2 ta Linear)
class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(16 * 2 * 2, 32), nn.ReLU(),
nn.Linear(32, 10))

    def forward(self, x):
        return self.tasnif(self.belgilar(x))

def oqit(model, epoxa=3, lr=3e-3):
    """Oddiy o'qitish sikli: Adam, paket 64, har epoxadan keyin test aniqligi."""
    opt = torch.optim.Adam(model.parameters(), lr); lossf = nn.CrossEntropyLoss()
    for ep in range(epoxa):
        model.train(); perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), 64):
            idx = perm[i:i + 64]
            opt.zero_grad(); loss = lossf(model(Xtr[idx]), ytr[idx]); loss.backward();
opt.step()
        model.eval()
        with torch.inference_mode():
```

```
            acc = (model(Xte).argmax(1) == yte).float().mean().item()
        print(f"  epoxa {ep + 1}: loss {loss.item():.3f}   test aniqlik {acc * 100:.1f} %")
    return acc

print("=== PyTorch FP32 etalon ===")
model = KichikCNN()
t0 = time.perf_counter(); acc_fp32 = oqit(model)
print(f"O'qitish vaqti: {time.perf_counter() - t0:.1f} s")
print(f"Parametrlar soni: {sum(p.numel() for p in model.parameters())}")

# Latency: batch 1, 300 takror (avval 30 ta isinish)
x1 = Xte[:1]
with torch.inference_mode():
    for _ in range(30): model(x1)
    t = []
    for _ in range(300):
        a = time.perf_counter(); model(x1); t.append(time.perf_counter() - a)
t = np.array(t) * 1e3
print(f"PyTorch eager, batch 1: p50 {np.percentile(t, 50):.3f} ms   p95 {np.percentile(t,
95):.3f} ms")

print("\n=== ONNX eksport ===")
# dynamic_axes — paket o'lchami o'zgaruvchan (keyingi qadamlarda batch 1/16/64 bilan sinaymiz)
# dynamo=False — TorchScript asosidagi eksport (sandbox'da onnxscript o'rnatilmagan)
torch.onnx.export(model, x1, "cnn_fp32.onnx", input_names=["input"], output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
                  opset_version=17, do_constant_folding=True, dynamo=False)
m = onnx.load("cnn_fp32.onnx"); onnx.checker.check_model(m)
print("Opset:", m.opset_import[0].version, "  tugunlar soni:", len(m.graph.node))
print("Operatorlar:", [n.op_type for n in m.graph.node])
print(f"Fayl hajmi: {os.path.getsize('cnn_fp32.onnx') / 1024:.1f} KB")
print("Kirish shakli:", [d.dim_param or d.dim_value for d in
m.graph.input[0].type.tensor_type.shape.dim])

# Moslik: PyTorch va ONNX Runtime chiqishlari bir xilmi?
sess = ort.InferenceSession("cnn_fp32.onnx", providers=["CPUExecutionProvider"])
with torch.inference_mode():
    ref = model(Xte[:64]).numpy()
out = sess.run(None, {"input": Xte[:64].numpy()})[0]
farq = np.abs(ref - out).max()
mos = np.allclose(ref, out, rtol=1e-3, atol=1e-4)
print(f"PyTorch vs ORT maksimal farq: {farq:.2e}  -> {'MOS' if mos else 'MOS EMAS'}")
acc_ort = (sess.run(None, {"input": Xte.numpy()})[0].argmax(1) == yte.numpy()).mean()
print(f"Test aniqlik: PyTorch {acc_fp32 * 100:.1f} %   ORT {acc_ort * 100:.1f} %")
print("\nIzoh: eval rejimida BatchNorm eksport paytida Conv ichiga birlashtirildi — ONNX
grafikda BN tuguni yo'q.")
print("GPU bo'lsa: xuddi shu fayl TensorRT (trtexec --onnx=cnn_fp32.onnx) va ORT
CUDAExecutionProvider'da ishlaydi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== PyTorch FP32 etalon ===`, `=== ONNX eksport ===`, `MOS`

TAJRIBA QILIB KO‘RING

- `model.eval()` ni eksportdan oldin olib tashlang (train rejimida) — farq va tugunlar qanday o'zgaradi?
- Kanallar sonini 8/16 dan 32/64 ga oshiring: parametrlar, hajm va latency qanday o'sadi?
- `opset_version=13` bilan eksport qiling va operatorlar ro'yxatini solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. ONNX Runtime: optimallashtirish darajalari, paket, oqimlar, Little qonuni

MAQSAD

ONNX Runtime'ning 4 ta grafik optimallashtirish darajasi (DISABLE_ALL, BASIC, EXTENDED, ALL) batch 1 da o'lchanadi va optimallashtirilgan grafikdagi tugunlar soni sanaladi; keyin paket 1/16/64/256 uchun throughput va Little qonuni (N = X·R), intra_op oqimlar 1 va 2 taqqoslanadi (GPU parallelligi o'rniga). Oxirida 2000 so'rov/s talabi uchun nusxalar soni rejalashtiriladi.

> **Kutilgan natija:** ORT ALL PyTorch eager'dan 6–9 marta tez; EXTENDED/ALL da tugunlar 10 → 7–8; batch 256 da throughput 10+ marta yuqori, latency ham oshadi; 2 oqim ~1,9x.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. ONNX Runtime: grafik optimallashtirish darajalari, paket o'lchami, oqimlar soni va
Little qonuni
import warnings; warnings.filterwarnings("ignore")
import time, math, numpy as np, pandas as pd, torch, torch.nn as nn, onnx, onnxruntime as ort

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# Latency vaznlarga bog'liq emas, shuning uchun bu qadamda model o'qitilmaydi — faqat
arxitektura.
class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
10))
    def forward(self, x):
        return self.tasnif(self.belgilar(x))

model = KichikCNN().eval()
x1 = torch.randn(1, 1, 8, 8)
torch.onnx.export(model, x1, "cnn_fp32.onnx", input_names=["input"], output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)

def olch(fn, takror=300, isinish=30):
    """fn() ni takror marta chaqirib, kechikish taqsimotini (ms) qaytaradi."""
    for _ in range(isinish): fn()
    t = []
    for _ in range(takror):
        a = time.perf_counter(); fn(); t.append(time.perf_counter() - a)
    t = np.array(t) * 1e3
    return dict(p50=np.percentile(t, 50), p95=np.percentile(t, 95), p99=np.percentile(t, 99),
mean=t.mean())

def ort_sess(opt_level, threads=2, saqla=None):
    so = ort.SessionOptions()
    so.graph_optimization_level = opt_level
    so.intra_op_num_threads = threads
    so.log_severity_level = 3                              # faqat xatolar
```

```
    if saqla: so.optimized_model_filepath = saqla        # optimallashtirilgan grafikni faylga
yozib tugunlarni sanaymiz
    return ort.InferenceSession("cnn_fp32.onnx", so, providers=["CPUExecutionProvider"])

print("=== ONNX Runtime o'lchovlari ===")
print("Mavjud provayderlar:", ort.get_available_providers(), "(GPU yo'q -> faqat CPU)")
L = ort.GraphOptimizationLevel
qatorlar = []
# 0) PyTorch eager etalon
with torch.inference_mode():
    r = olch(lambda: model(x1))
qatorlar.append(dict(dvigatel="PyTorch eager", opt="-", tugun="-", **{k: round(v, 3) for k, v
in r.items()}))
# 1) ORT 4 ta optimallashtirish darajasi, batch 1
xn = x1.numpy()
for nomi, lvl in (("DISABLE_ALL", L.ORT_DISABLE_ALL), ("BASIC", L.ORT_ENABLE_BASIC),
                  ("EXTENDED", L.ORT_ENABLE_EXTENDED), ("ALL", L.ORT_ENABLE_ALL)):
    s = ort_sess(lvl, saqla=f"opt_{nomi}.onnx")
    tugun = len(onnx.load(f"opt_{nomi}.onnx").graph.node)
    r = olch(lambda: s.run(None, {"input": xn}))
    qatorlar.append(dict(dvigatel="ORT CPU", opt=nomi, tugun=tugun, **{k: round(v, 3) for k, v
in r.items()}))
df = pd.DataFrame(qatorlar)
print("\nBatch 1, 2 oqim (ms):"); print(df.to_string(index=False))
tez = df.loc[0, "p50"] / df.loc[4, "p50"]
print(f"ORT ALL PyTorch eager'dan {tez:.1f} marta tez (Python xarajati yo'q, grafik oldindan
tuzilgan).")

# 2) Paket o'lchami va throughput (Little qonuni: tizimdagi so'rovlar = throughput x latency)
print("\n--- Paket o'lchami (ORT ALL, 2 oqim) ---")
s = ort_sess(L.ORT_ENABLE_ALL)
qat = []
for b in (1, 16, 64, 256):
    xb = np.random.rand(b, 1, 8, 8).astype(np.float32)
    r = olch(lambda: s.run(None, {"input": xb}), takror=100)
    thr = b / (r["mean"] / 1e3)
    qat.append(dict(batch=b, p50_ms=round(r["p50"], 3), p99_ms=round(r["p99"], 3),
throughput=round(thr),
                    little_N=round(thr * r["mean"] / 1e3, 1)))
dfb = pd.DataFrame(qat); print(dfb.to_string(index=False))
print("little_N = throughput x latency = paketdagi so'rovlar soni (Little qonuni sinxron siklda
aynan batch'ga teng).")

# 3) Oqimlar soni: 1 va 2 (GPU parallelligi o'rniga CPU parallelligi)
print("\n--- intra_op oqimlar: 1 vs 2 (batch 256) ---")
xb = np.random.rand(256, 1, 8, 8).astype(np.float32)
nat = {}
for th in (1, 2):
    s = ort_sess(L.ORT_ENABLE_ALL, threads=th)
    nat[th] = olch(lambda: s.run(None, {"input": xb}), takror=100)["mean"]
    print(f"  {th} oqim: o'rtacha {nat[th]:.3f} ms  -> {256 / nat[th] * 1e3:,.0f} rasm/s")
print(f"  Tezlanish 2 oqimda: {nat[1] / nat[2]:.2f}x (ideal 2.0x; farq — sinxronizatsiya va
kichik model)")

# 4) Rejalashtirish: 2000 so'rov/s va p99 <= 2 ms talabi uchun nechta parallel nusxa kerak?
lam, p99 = 2000, df.loc[4, "p99"]
print(f"\nLittle qonuni bo'yicha reja ({lam} so'rov/s, p99 bo'yicha):")
for nomi, lat in (("bizning CNN", p99), ("ResNet-50 CPU (~20 ms)", 20.0), ("ResNet-50 T4 INT8
(~0.4 ms/rasm)", 0.4)):
    N = lam * lat / 1e3
    print(f"  {nomi:<34} {lam} x {lat:.3f} ms = {N:6.1f} parallel ishlov -> kamida
{math.ceil(N)} nusxa/oqim")
```

```
print("GPU'da: ORT CUDAExecutionProvider / TensorRT'da batch 32 da throughput 5-10 marta
o'sadi, latency 10+ marta oshadi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== ONNX Runtime o'lchovlari ===`, `Paket o'lchami`, `Little qonuni`

TAJRIBA QILIB KO‘RING

- `takror` ni 300 dan 2000 ga oshiring — p99 barqarorlashadimi?
- `inter_op_num_threads` va `execution_mode = ORT_PARALLEL` qo'shib ko'ring.
- Little qonuni rejasiga p50 o'rniga p99 ni emas, o'rtachani qo'ysangiz nusxalar soni qanday o'zgaradi va nega p99 to'g'riroq?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. PyTorch'da INT8 dinamik kvantlash (Linear)

MAQSAD

MLP (Linear og'ir model) `torch.ao.quantization.quantize_dynamic` bilan INT8 ga o'tkaziladi: kvantlash formulasi q = round(w/s) qo'lda tekshiriladi, hajm (state_dict baytlari), aniqlik va batch 1/256 kechikishi FP32 bilan solishtiriladi. Sandbox CPU'da AVX yo'qligi sababli `qnnpack` dvigateli majburan tanlanadi — real serverda bu qator kerak emas.

> **Kutilgan natija:** Hajm ~3,8 marta kichik (335 → 89 KB), aniqlik o'zgarmaydi (±0,3 p.p.), tezlik bu CPU'da 0,7–1,0x (VNNI yo'q).

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. PyTorch'da dinamik INT8 kvantlash (Linear qatlamlar): hajm, aniqlik, tezlik
import warnings; warnings.filterwarnings("ignore")
import io, time, numpy as np, pandas as pd, torch, torch.nn as nn
from torch.ao.quantization import quantize_dynamic
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)
# Sandbox CPU'da AVX/VNNI yo'q: fbgemm/onednn dvigatellari ishlamaydi, qnnpack (ARM/eski x86
uchun) ishlaydi.
# Haqiqiy serverda (AVX-512 VNNI) bu qator kerak emas — 'x86' dvigateli avtomatik tanlanadi.
torch.backends.quantized.engine = "qnnpack"
print("Kvantlash dvigateli:", torch.backends.quantized.engine, "| mavjud:",
torch.backends.quantized.supported_engines)

X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype(np.float32)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Xtr, Xte, ytr, yte = map(torch.tensor, (Xtr, Xte, ytr, yte))

# MLP — Linear (MatMul) og'ir model: dinamik kvantlash aynan shunday modellarga mo'ljallangan
(BERT, LSTM kabi)
model = nn.Sequential(nn.Linear(64, 256), nn.ReLU(), nn.Linear(256, 256), nn.ReLU(),
nn.Linear(256, 10))
opt = torch.optim.Adam(model.parameters(), 2e-3); lossf = nn.CrossEntropyLoss()
for ep in range(3):
    model.train(); perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lossf(model(Xtr[idx]), ytr[idx]).backward();
opt.step()
model.eval()

def aniqlik(m):
    with torch.inference_mode():
        return (m(Xte).argmax(1) == yte).float().mean().item() * 100

def hajm_kb(m):
    b = io.BytesIO(); torch.save(m.state_dict(), b); return b.tell() / 1024

def latency(m, x, takror=300):
    with torch.inference_mode():
        for _ in range(30): m(x)
        t = []
```

```
        for _ in range(takror):
            a = time.perf_counter(); m(x); t.append(time.perf_counter() - a)
    return np.percentile(np.array(t) * 1e3, 50)

print("\n=== PyTorch INT8 dinamik kvantlash ===")
qmodel = quantize_dynamic(model, {nn.Linear}, dtype=torch.qint8)   # vaznlar INT8,
faollashuvlar ish vaqtida kvantlanadi
print("Kvantlangan model tuzilmasi:")
for nomi, mod in qmodel.named_children():
    print(f"  {nomi}: {repr(mod)}")

# Kvantlash formulasini qo'lda tekshiramiz: q = round(w / s), s = max|w| / 127 (simmetrik, per-
tensor)
w = model[0].weight.detach()
s = w.abs().max() / 127
q = torch.round(w / s).clamp(-128, 127)
print(f"\n1-qatlam vazni: diapazon [{w.min():.3f}, {w.max():.3f}]  masshtab s = {s:.5f}")
print(f"  INT8 -> FP32 qayta tiklash xatosi: max {((q * s) - w).abs().max():.5f}, o'rtacha {((q
* s) - w).abs().mean():.6f}")
print(f"  Noyob qiymatlar: FP32 {w.unique().numel()} ta -> INT8 {q.unique().numel()} ta")

x1, xb = Xte[:1], Xte[:256]
rows = []
for nomi, m in (("FP32", model), ("INT8 dinamik", qmodel)):
    rows.append(dict(model=nomi, hajm_KB=round(hajm_kb(m), 1), aniqlik=round(aniqlik(m), 2),
                     p50_batch1_ms=round(latency(m, x1), 3), p50_batch256_ms=round(latency(m,
xb, 100), 3)))
df = pd.DataFrame(rows); print("\n" + df.to_string(index=False))
print(f"\nHajm {df.hajm_KB[0] / df.hajm_KB[1]:.1f} marta kichraydi (nazariy chegara 4x; bias va
masshtablar FP32 da qoladi).")
print(f"Aniqlik farqi: {df.aniqlik[1] - df.aniqlik[0]:+.2f} p.p.   Tezlik (batch 256):
{df.p50_batch256_ms[0] / df.p50_batch256_ms[1]:.2f}x")
print("Izoh: bu CPU'da VNNI buyruqlari yo'q, shuning uchun INT8 tezlanishi kichik yoki hatto
salbiy bo'lishi mumkin —")
print("  bu A3 matnidagi 5-xato ('VNNI yo'qligini tekshirmaslik') ning jonli namunasi. VNNI/AMX
CPU'da 2-3x kutiladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== PyTorch INT8 dinamik kvantlash ===`, `DynamicQuantizedLinear`, `Hajm`

TAJRIBA QILIB KO‘RING

- Qatlam kengligini 256 dan 1024 ga oshiring — hajm nisbati 4x ga yaqinlashadimi, tezlik nisbati o'zgaradimi?
- `dtype=torch.float16` bilan `quantize_dynamic` chaqirib ko'ring (CPU'da qo'llab-quvvatlanadimi?).
- Per-channel masshtab bilan qo'lda kvantlab, qayta tiklash xatosini per-tensor bilan solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. ONNX Runtime'da kvantlash: dinamik va statik (MinMax, Percentile, shovqin)

MAQSAD

CNN ONNX modeli `quant_pre_process` dan o'tkazilib, `quantize_dynamic` va `quantize_static` (QDQ format) bilan kvantlanadi: per-channel MinMax, per-channel Percentile, per-tensor MinMax va ataylab noto'g'ri — tasodifiy shovqin bilan kalibrlash (A3 matnidagi 3-xato). Har variant uchun hajm, aniqlik, p50, throughput va Q/DQ tugunlar soni jadvalga yig'iladi.

> **Kutilgan natija:** Hajm 17 → 11–13 KB; real kalibrlash aniqlikni saqlaydi (±0,3 p.p.), shovqinli kalibrlash 1 p.p. atrofida yo'qotadi; INT8 bu CPU'da tezroq emas.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. ONNX Runtime'da kvantlash: dinamik va statik (MinMax, Percentile, shovqin bilan
kalibrlash)
import warnings; warnings.filterwarnings("ignore")
import io, os, time, logging, contextlib, numpy as np, pandas as pd, torch, torch.nn as nn,
onnx, onnxruntime as ort
from onnxruntime.quantization import (quantize_dynamic, quantize_static, QuantType,
QuantFormat,
                                      CalibrationDataReader, CalibrationMethod)
from onnxruntime.quantization.shape_inference import quant_pre_process
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

logging.disable(logging.WARNING)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)

X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype(np.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Ttr, Tte, Tytr = torch.tensor(Xtr), torch.tensor(Xte), torch.tensor(ytr)

class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
10))
    def forward(self, x):
        return self.tasnif(self.belgilar(x))

# O'qitish (2 epoxa) va eksport
model = KichikCNN(); opt = torch.optim.Adam(model.parameters(), 3e-3); lossf =
nn.CrossEntropyLoss()
for ep in range(3):
    model.train(); perm = torch.randperm(len(Ttr))
    for i in range(0, len(Ttr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lossf(model(Ttr[idx]), Tytr[idx]).backward();
opt.step()
model.eval()
```

```
torch.onnx.export(model, Tte[:1], "cnn_fp32.onnx", input_names=["input"], output_names=
["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)
quant_pre_process("cnn_fp32.onnx", "cnn_prep.onnx")       # shakl inferensi + optimallashtirish
(kvantlash sifati uchun)

class Reader(CalibrationDataReader):
    """Kalibrlash to'plami: ORT har safar bitta {kirish: massiv} lug'at so'raydi."""
    def __init__(self, arr, paket=32):
        self.it = iter([{"input": arr[i:i + paket]} for i in range(0, len(arr), paket)])
    def get_next(self):
        return next(self.it, None)

def statik(chiqish, kalib, usul, per_channel=True):
    with contextlib.redirect_stdout(io.StringIO()):               # kalibrator loglarini
yashiramiz
        quantize_static("cnn_prep.onnx", chiqish, Reader(kalib), quant_format=QuantFormat.QDQ,
                        activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8,
                        per_channel=per_channel, calibrate_method=usul)

print("=== ONNX Runtime kvantlash ===")
kalib = Xtr[:320]                                                  # 320 ta real namuna
shovqin = (np.random.rand(320, 1, 8, 8) * 8).astype(np.float32)     # 3-xato: real taqsimotga
mos kelmaydigan shovqin (0..8, real 0..1)
quantize_dynamic("cnn_prep.onnx", "cnn_dyn.onnx", weight_type=QuantType.QInt8)
statik("cnn_s_minmax.onnx", kalib, CalibrationMethod.MinMax)
statik("cnn_s_pct.onnx", kalib, CalibrationMethod.Percentile)
statik("cnn_s_pt.onnx", kalib, CalibrationMethod.MinMax, per_channel=False)
statik("cnn_s_noise.onnx", shovqin, CalibrationMethod.MinMax)

def baho(fayl):
    so = ort.SessionOptions(); so.intra_op_num_threads = 2
    s = ort.InferenceSession(fayl, so, providers=["CPUExecutionProvider"])
    acc = (s.run(None, {"input": Xte})[0].argmax(1) == yte).mean() * 100
    x1 = Xte[:1]
    for _ in range(30): s.run(None, {"input": x1})
    t = []
    for _ in range(300):
        a = time.perf_counter(); s.run(None, {"input": x1}); t.append(time.perf_counter() - a)
    xb = Xte[:256]
    for _ in range(5): s.run(None, {"input": xb})
    a = time.perf_counter()
    for _ in range(20): s.run(None, {"input": xb})
    thr = 256 * 20 / (time.perf_counter() - a)
    ops = [n.op_type for n in onnx.load(fayl).graph.node]
    return dict(hajm_KB=round(os.path.getsize(fayl) / 1024, 1), aniqlik=round(acc, 2),
                p50_ms=round(np.percentile(np.array(t) * 1e3, 50), 3), throughput=round(thr),
                QDQ=sum(o in ("QuantizeLinear", "DequantizeLinear", "DynamicQuantizeLinear")
for o in ops))

rows = []
for nomi, f in (("FP32", "cnn_prep.onnx"), ("Dinamik INT8", "cnn_dyn.onnx"),
                ("Statik, per-channel, MinMax", "cnn_s_minmax.onnx"),
                ("Statik, per-channel, Percentile", "cnn_s_pct.onnx"),
                ("Statik, per-tensor, MinMax", "cnn_s_pt.onnx"),
                ("Statik, SHOVQIN bilan kalibrlash", "cnn_s_noise.onnx")):
    rows.append(dict(konfiguratsiya=nomi, **baho(f)))
df = pd.DataFrame(rows)
df["farq_pp"] = (df.aniqlik - df.aniqlik[0]).round(2)
print(df.to_string(index=False))
print("\nOperatorlar (dinamik):", [n.op_type for n in onnx.load("cnn_dyn.onnx").graph.node]
[:9], "...")
```

```
print("Operatorlar (statik QDQ):", [n.op_type for n in
onnx.load("cnn_s_minmax.onnx").graph.node][:8], "...")
print("\nXulosa: statik kvantlash kalibrlash to'plamiga bog'liq — noto'g'ri taqsimotli shovqin
bilan kalibrlangan model aniqlikni yo'qotadi;")
print("  per-tensor vs per-channel farqi kichik modelda sezilmasligi mumkin, ResNet-50 da 1-2
p.p. beradi.")
print("  INT8 tezlanishi bu CPU'da (VNNI yo'q) kutilmaydi; VNNI/AMX serverda statik INT8 2-4x
beradi, GPU'da TensorRT INT8 ishlatiladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== ONNX Runtime kvantlash ===`, `SHOVQIN`, `Operatorlar (statik QDQ)`

TAJRIBA QILIB KO‘RING

- Kalibrlash to'plamini 320 dan 32 va 1000 ga o'zgartiring — aniqlik qanday?
- `CalibrationMethod.Entropy` qo'shing va uch usulni solishtiring.
- `nodes_to_exclude` bilan birinchi Conv'ni FP32 da qoldiring (sezgir qatlam) — aniqlik tiklanadimi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Yakuniy jadval, Pareto tahlili va grafik

MAQSAD

6 ta konfiguratsiya (PyTorch FP32, PyTorch INT8 dinamik, ORT FP32, ORT INT8 dinamik, ORT statik MinMax/Percentile) bitta jadvalga yig'iladi: hajm, aniqlik, p50 (batch 1), throughput (batch 256), tezlanish. Pareto-optimal nuqtalar (latency kichik, aniqlik katta) aniqlanadi va ikki o'qli grafik chiziladi: latency–aniqlik va hajm–throughput.

> **Kutilgan natija:** ORT variantlari PyTorch'dan 4–7 marta tez; Pareto chegarasida ORT FP32 va statik Percentile; grafikda qizil nuqtalar — Pareto.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Yakuniy jadval: aniqlik vs tezlik vs hajm, Pareto tahlili va grafik
import warnings; warnings.filterwarnings("ignore")
import io, os, time, logging, contextlib, numpy as np, pandas as pd, torch, torch.nn as nn,
onnxruntime as ort
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from torch.ao.quantization import quantize_dynamic as torch_qdyn
from onnxruntime.quantization import (quantize_dynamic, quantize_static, QuantType,
QuantFormat,
                                      CalibrationDataReader, CalibrationMethod)
from onnxruntime.quantization.shape_inference import quant_pre_process
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

logging.disable(logging.WARNING)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)
torch.backends.quantized.engine = "qnnpack"      # sandbox CPU'da AVX yo'q (3-qadamga qarang)

X, y = load_digits(return_X_y=True)
X = (X / 16.0).astype(np.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
Ttr, Tte, Tytr, Tyte = torch.tensor(Xtr), torch.tensor(Xte), torch.tensor(ytr),
torch.tensor(yte)

class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
10))
    def forward(self, x):
        return self.tasnif(self.belgilar(x))

model = KichikCNN(); opt = torch.optim.Adam(model.parameters(), 3e-3); lossf =
nn.CrossEntropyLoss()
for ep in range(3):
    model.train(); perm = torch.randperm(len(Ttr))
    for i in range(0, len(Ttr), 64):
        idx = perm[i:i + 64]; opt.zero_grad(); lossf(model(Ttr[idx]), Tytr[idx]).backward();
opt.step()
model.eval()
```

```
torch.onnx.export(model, Tte[:1], "cnn_fp32.onnx", input_names=["input"], output_names=
["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)
quant_pre_process("cnn_fp32.onnx", "cnn_prep.onnx")

class Reader(CalibrationDataReader):
    def __init__(self, arr, paket=32):
        self.it = iter([{"input": arr[i:i + paket]} for i in range(0, len(arr), paket)])
    def get_next(self):
        return next(self.it, None)

quantize_dynamic("cnn_prep.onnx", "cnn_dyn.onnx", weight_type=QuantType.QInt8)
for nomi, usul in (("minmax", CalibrationMethod.MinMax), ("pct",
CalibrationMethod.Percentile)):
    with contextlib.redirect_stdout(io.StringIO()):
        quantize_static("cnn_prep.onnx", f"cnn_s_{nomi}.onnx", Reader(Xtr[:320]),
quant_format=QuantFormat.QDQ,
                        activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8,
per_channel=True, calibrate_method=usul)

def olch(fn, x1, xb):
    """p50 latency (batch 1, ms) va throughput (batch 256, rasm/s)."""
    for _ in range(30): fn(x1)
    t = []
    for _ in range(300):
        a = time.perf_counter(); fn(x1); t.append(time.perf_counter() - a)
    for _ in range(5): fn(xb)
    a = time.perf_counter()
    for _ in range(20): fn(xb)
    return np.percentile(np.array(t) * 1e3, 50), 256 * 20 / (time.perf_counter() - a)

def torch_hajm(m):
    b = io.BytesIO(); torch.save(m.state_dict(), b); return b.tell() / 1024

rows = []
# PyTorch variantlari
qmodel = torch_qdyn(model, {nn.Linear}, dtype=torch.qint8)
for nomi, m in (("PyTorch FP32", model), ("PyTorch INT8 dinamik (Linear)", qmodel)):
    with torch.inference_mode():
        acc = (m(Tte).argmax(1) == Tyte).float().mean().item() * 100
        p50, thr = olch(lambda x: m(x), Tte[:1], Tte[:256])
    rows.append(dict(konfiguratsiya=nomi, hajm_KB=round(torch_hajm(m), 1), aniqlik=round(acc,
2), p50_ms=round(p50, 3), throughput=round(thr)))
# ONNX Runtime variantlari
for nomi, f in (("ORT FP32 (ALL)", "cnn_prep.onnx"), ("ORT INT8 dinamik", "cnn_dyn.onnx"),
                ("ORT INT8 statik MinMax", "cnn_s_minmax.onnx"), ("ORT INT8 statik Percentile",
"cnn_s_pct.onnx")):
    so = ort.SessionOptions(); so.intra_op_num_threads = 2
    s = ort.InferenceSession(f, so, providers=["CPUExecutionProvider"])
    acc = (s.run(None, {"input": Xte})[0].argmax(1) == yte).mean() * 100
    p50, thr = olch(lambda x: s.run(None, {"input": x}), Xte[:1], Xte[:256])
    rows.append(dict(konfiguratsiya=nomi, hajm_KB=round(os.path.getsize(f) / 1024, 1),
aniqlik=round(acc, 2), p50_ms=round(p50, 3), throughput=round(thr)))

df = pd.DataFrame(rows)
df["tezlanish"] = (df.p50_ms[0] / df.p50_ms).round(2)
print("=== Yakuniy jadval ===")
print(df.to_string(index=False))

# Pareto: latency kichik, aniqlik katta — hech kim ikkala ko'rsatkichda ham yaxshiroq bo'lmagan
nuqtalar
def pareto(df):
```

```
    opt = []
    for i, r in df.iterrows():
        ustun = ((df.p50_ms <= r.p50_ms) & (df.aniqlik >= r.aniqlik) & ((df.p50_ms < r.p50_ms)
| (df.aniqlik > r.aniqlik))).any()
        if not ustun: opt.append(r.konfiguratsiya)
    return opt
print("\n=== Pareto tahlili ===")
par = pareto(df)
print("Pareto-optimal (latency-aniqlik):", par)
eng_tez = df.loc[df.p50_ms.idxmin()]; eng_kichik = df.loc[df.hajm_KB.idxmin()]
print(f"Eng tez: {eng_tez.konfiguratsiya} ({eng_tez.p50_ms} ms, {eng_tez.tezlanish}x);  eng
kichik: {eng_kichik.konfiguratsiya} ({eng_kichik.hajm_KB} KB)")
print("Tavsiya: tibbiy tasniflash (aniqlik birinchi) -> ORT FP32; chekka qurilma (hajm/energiya
birinchi) -> statik INT8;")
print("  server CPU'da VNNI bo'lsa statik INT8 ham tezroq bo'ladi; GPU'da TensorRT FP16 odatda
Pareto chegarasida.")

# Grafik: ikki o'q — latency vs aniqlik, hajm vs throughput
fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
for _, r in df.iterrows():
    rang = "tab:red" if r.konfiguratsiya in par else "tab:blue"
    ax[0].scatter(r.p50_ms, r.aniqlik, s=70, color=rang); ax[0].annotate(r.konfiguratsiya,
(r.p50_ms, r.aniqlik), fontsize=7, xytext=(4, 3), textcoords="offset points")
    ax[1].scatter(r.hajm_KB, r.throughput, s=70, color=rang); ax[1].annotate(r.konfiguratsiya,
(r.hajm_KB, r.throughput), fontsize=7, xytext=(4, 3), textcoords="offset points")
ax[0].set_xlabel("p50 latency, ms (batch 1)"); ax[0].set_ylabel("Test aniqlik, %");
ax[0].set_title("Latency - aniqlik (qizil = Pareto)"); ax[0].set_xscale("log");
ax[0].grid(alpha=.3)
ax[1].set_xlabel("Model hajmi, KB"); ax[1].set_ylabel("Throughput, rasm/s (batch 256)");
ax[1].set_title("Hajm - throughput"); ax[1].grid(alpha=.3)
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Yakuniy jadval ===`, `=== Pareto tahlili ===`, `Pareto-optimal`

TAJRIBA QILIB KO‘RING

- Pareto mezonini (hajm, aniqlik) juftligiga o'zgartiring — qaysi konfiguratsiyalar qoladi?
- Uchinchi o'q sifatida hajmni nuqta kattaligi bilan ko'rsating.
- O'qitish epoxasini 1 ga tushiring: aniqlik farqlari kvantlash tufaylimi yoki o'qitish shovqinimi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Energiya bahosi, nusxalar rejasi va GPU/TensorRT yo'l xaritasi

MAQSAD

RAPL/nvidia-smi yo'qligi sababli quvvat MODEL bilan baholanadi: P = P_bosh + (P_to'liq − P_bosh)·CPU_bandlik, bandlik `time.process_time()/devor vaqti` dan olinadi. FP32 va INT8 dinamik, 1 va 2 oqim, batch 1 va 64 uchun µJ/inferens (to'liq va dinamik) hamda inferens/s/W hisoblanadi; Little qonuni bilan 5000 so'rov/s uchun nusxalar va quvvat; oxirida GPU'da TensorRT FP16/INT8 va quvvat o'lchash qadamlari (buyruqlar va kutilgan raqamlar).

> **Kutilgan natija:** Batch 64 da energiya batch 1 dan 5–15 marta kam; 2 oqim batch 1 da energiya ortadi (bandlik 2x, tezlik oshmaydi); ustunli grafik.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Energiya bahosi (CPU vaqti asosida), nusxalar rejasi va GPU/TensorRT yo'l xaritasi
import warnings; warnings.filterwarnings("ignore")
import time, math, logging, numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as
ort
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
from onnxruntime.quantization import quantize_dynamic, QuantType
from onnxruntime.quantization.shape_inference import quant_pre_process

logging.disable(logging.WARNING)
torch.manual_seed(0); torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# Energiya latency kabi vaznlarga bog'liq emas — modelni o'qitmaymiz
class KichikCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.belgilar = nn.Sequential(
            nn.Conv2d(1, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2))
        self.tasnif = nn.Sequential(nn.Flatten(), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32,
10))
    def forward(self, x):
        return self.tasnif(self.belgilar(x))

model = KichikCNN().eval(); x1 = torch.randn(1, 1, 8, 8)
torch.onnx.export(model, x1, "cnn_fp32.onnx", input_names=["input"], output_names=["logits"],
                  dynamic_axes={"input": {0: "batch"}, "logits": {0: "batch"}},
opset_version=17, dynamo=False)
quant_pre_process("cnn_fp32.onnx", "cnn_prep.onnx")
quantize_dynamic("cnn_prep.onnx", "cnn_dyn.onnx", weight_type=QuantType.QInt8)

# --- Quvvat modeli. Sandbox'da RAPL/nvidia-smi yo'q, shuning uchun quvvatni MODEL bilan
baholaymiz:
#     P = P_bosh + (P_tolik - P_bosh) * CPU_bandlik.  Real labda bu qiymatlar RAPL (CPU) yoki
NVML (GPU) dan o'qiladi.
P_TOLIK_W = 15.0      # 2 yadro to'liq yuklangan server CPU ulushi (taxmin; o'zgartirib
ko'ring)
P_BOSH_W = 5.0        # bo'sh holat

def energiya(fn, batch, takror):
    for _ in range(20): fn()
```

```
    w0, c0 = time.perf_counter(), time.process_time()      # devor vaqti va CPU vaqti (barcha
oqimlar yig'indisi)
    for _ in range(takror): fn()
    wall, cpu = time.perf_counter() - w0, time.process_time() - c0
    bandlik = min(cpu / wall / 2, 1.0)                        # 2 yadroga nisbatan
    P = P_BOSH_W + (P_TOLIK_W - P_BOSH_W) * bandlik
    n = batch * takror
    return dict(latency_ms=wall / takror * 1e3, throughput=n / wall, cpu_bandlik=bandlik * 100,
                P_W=P, uJ_tolik=P * wall / n * 1e6, uJ_dinamik=(P - P_BOSH_W) * wall / n * 1e6,
inf_s_W=n / wall / P)

print("=== Energiya bahosi ===")
rows = []
for nomi, fayl in (("ORT FP32", "cnn_prep.onnx"), ("ORT INT8 dinamik", "cnn_dyn.onnx")):
    for th in (1, 2):
        so = ort.SessionOptions(); so.intra_op_num_threads = th
        s = ort.InferenceSession(fayl, so, providers=["CPUExecutionProvider"])
        for b in (1, 64):
            xb = np.random.rand(b, 1, 8, 8).astype(np.float32)
            r = energiya(lambda: s.run(None, {"input": xb}), b, 200 if b == 1 else 60)
            rows.append(dict(konfiguratsiya=nomi, oqim=th, batch=b, **{k: round(v, 3 if k ==
"latency_ms" else 1) for k, v in r.items()}))
df = pd.DataFrame(rows); print(df.to_string(index=False))
eng = df.loc[df.uJ_tolik.idxmin()]
print(f"\nEng tejamkor: {eng.konfiguratsiya}, {eng.oqim} oqim, batch {eng.batch}:
{eng.uJ_tolik} uJ/inferens (to'liq; 1 mJ = 1000 uJ), {eng.inf_s_W} inferens/s/W")
print("Kuzatuv: batch 64 da inferens boshiga energiya batch 1 ga nisbatan ancha kam — doimiy
xarajat (Python, sessiya chaqiruvi) taqsimlanadi.")
print("  uJ_tolik — TCO uchun (bo'sh quvvat ham hisobga olinadi), uJ_dinamik — algoritm
samaradorligi uchun. ResNet-50 da bu 10-100 mJ bo'ladi.")

# --- Little qonuni bo'yicha rejalashtirish
print("\n=== Nusxalar rejasi (Little qonuni) ===")
lam = 5000                                                  # kerakli yuklama, so'rov/s
rejalar = [(r.konfiguratsiya, r.latency_ms, r.P_W) for _, r in df[(df.batch == 1) & (df.oqim ==
2)].iterrows()]
rejalar.append(("ResNet-50 FP32 CPU (taxmin 20 ms, 15 W)", 20.0, 15.0))
rejalar.append(("ResNet-50 INT8 VNNI CPU (taxmin 8 ms, 15 W)", 8.0, 15.0))
for nomi, lat, P in rejalar:
    N = lam * lat / 1e3
    print(f"  {nomi:<44} {lam} x {lat:.3f} ms = {N:7.1f} parallel -> {math.ceil(N):4d} nusxa (2
yadro), quvvat ~{math.ceil(N) * P:.0f} W")

# --- GPU/TensorRT'da bu qadam qanday bo'ladi (sandbox'da GPU yo'q — faqat yo'l xaritasi)
print("\n=== GPU'da bu qadam qanday bo'ladi ===")
gpu = pd.DataFrame([
    dict(bosqich="Dvigatel qurish", buyruq="trtexec --onnx=cnn_fp32.onnx --saveEngine=fp32.plan
--minShapes=input:1x1x8x8 --optShapes=input:32x1x8x8 --maxShapes=input:256x1x8x8"),
    dict(bosqich="FP16", buyruq="trtexec ... --fp16   (Tensor Core, ResNet-50 da ~2.7x, aniqlik
-0.1 p.p.)"),
    dict(bosqich="INT8", buyruq="trtexec --onnx=cnn_s_pct.onnx --int8 --fp16   (Q/DQ tugunli
ONNX kalibrlashsiz qabul qilinadi)"),
    dict(bosqich="Quvvat", buyruq="nvidia-smi --query-gpu=power.draw --format=csv -lms 100
(parallel oqimda, 10-30 s davomida o'rtacha)"),
    dict(bosqich="Energiya", buyruq="mJ/rasm = P_ortacha / throughput * 1000;  rasm/s/W =
throughput / P_ortacha"),
])
for _, r in gpu.iterrows():
    print(f"  {r.bosqich:<16} {r.buyruq}")
print("  Kutilgan (T4, ResNet-50, batch 8): FP32 119 mJ -> FP16 45 mJ -> INT8 27 mJ/rasm;
quvvat deyarli o'zgarmaydi (~70 W), energiya throughput'ga proporsional.")

# Grafik: mJ/inferens ustunlari
```

```
fig, ax = plt.subplots(figsize=(8, 4))
lab = [f"{r.konfiguratsiya}\n{r.oqim} oqim, b={r.batch}" for _, r in df.iterrows()]
ax.bar(range(len(df)), df.uJ_tolik, color=["tab:blue" if "FP32" in l else "tab:orange" for l in
lab], label="to'liq")
ax.bar(range(len(df)), df.uJ_dinamik, color="k", alpha=.35, width=.4, label="dinamik")
ax.set_xticks(range(len(df))); ax.set_xticklabels(lab, fontsize=7); ax.set_ylabel("uJ /
inferens (model bo'yicha)")
ax.set_yscale("log"); ax.set_title("Energiya bahosi: to'liq va dinamik"); ax.legend();
ax.grid(axis="y", alpha=.3)
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Energiya bahosi ===`, `=== Nusxalar rejasi (Little qonuni) ===`, `=== GPU'da bu qadam qanday bo'ladi ===`

TAJRIBA QILIB KO‘RING

- P_TOLIK_W va P_BOSH_W ni o'z noutbukingiz TDP'siga moslang — xulosa o'zgaradimi?
- `psutil.Process().cpu_times()` bilan user/system vaqtni ajrating.
- Batch 256 va 1024 qo'shing: µJ/inferens qachon to'yinadi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Opset va tugunlar** 1-qadam kodida modelni opset 13, 15 va 17 bilan eksport qiling. Har biri uchun tugunlar soni, operatorlar ro'yxati, fayl hajmi va PyTorch bilan maksimal farqni jadvalga yozing. Qaysi opset'da farq bor-yo'qligini izohlang.	oson	5
2	**Optimallashtirish darajalari jadvali** 2-qadamni `takror=1000` bilan 3 marta qayta bajaring va har daraja uchun p50/p95/p99 o'rtachasini va tarqalishini (min–max) hisoblang. Qaysi daraja barqaror tezlanish beradi? Natijani 4 qatorli jadval va 2–3 jumla xulosa bilan topshiring.	oson	5
3	**Kvantlash formulasi** 3-qadamdagi qo'lda kvantlash kodini asimmetrik variantga (s = (max−min)/255, z = round(−min/s)) o'zgartiring va simmetrik bilan qayta tiklash xatosini (max, o'rtacha) uchala Linear qatlam uchun solishtiring. Qaysi qatlamda farq katta va nega (vaznlar taqsimotini gistogramma bilan ko'rsating)?	oson	5
4	**Kalibrlash to'plami hajmi** 4-qadamda statik kvantlashni 8, 32, 128, 320 va 1000 namunali kalibrlash to'plamlari bilan (MinMax va Percentile) bajaring. Aniqlik va kvantlash vaqtini jadvalga yozing, aniqlik–hajm grafigini chizing va kalibrlash to'plamining minimal yetarli hajmini asoslang.	o'rta	10
5	**Sezgir qatlam tahlili** 4-qadam kodida `quantize_static` ning `nodes_to_exclude` parametri bilan har bir Conv/Gemm tugunini navbatma-navbat FP32 da qoldiring (qolganlari INT8). Har konfiguratsiya uchun aniqlik va hajmni o'lchang, eng sezgir qatlamni aniqlang va «INT8 + 1 ta FP32 qatlam» aralash variantini to'liq INT8 bilan solishtiring.	o'rta	10
6	**Paket va kechikish byudjeti** 2-qadamni batch 1, 2, 4, 8, 16, 32, 64, 128, 256 uchun bajarib, latency va throughput egri chiziqlarini chizing. p99 kechikish 0,1 ms dan oshmasligi talabi bilan maksimal paket o'lchamini toping va Little qonuni orqali 10 000 so'rov/s uchun kerakli nusxalar sonini ikki variantda (batch 1 va topilgan maksimal batch) hisoblang.	o'rta	10
7	**MLP vs CNN: dinamik kvantlash farqi** 3- va 4-qadamlar asosida MLP (Linear og'ir) va CNN (Conv og'ir) uchun dinamik INT8 kvantlashning hajm, aniqlik va tezlik ta'sirini bir xil metodika bilan o'lchang (PyTorch va ORT ikkalasida). A3 matnidagi «dinamik kvantlash MatMul og'ir modellarga mos» tezisini o'z raqamlaringiz bilan tasdiqlang yoki rad eting.	o'rta	10
8	**QAT bilan aniqlikni tiklash** 4-qadamdagi shovqin bilan kalibrlangan (yoki ataylab per-tensor, 4 bitli ta'sirni simulyatsiya qilish uchun vaznlarni round(w/s) bilan buzilgan) modelni oling va PyTorch'da soxta kvantlash (fake quantization: forward'da vaznlarni kvantlab-qayta tiklash, backward oddiy) bilan 2 epoxa qayta o'qiting. INT8 aniqligini FP32 ga 0,5 p.p. ichida yaqinlashtiring; o'qitish egri chizig'i va yakuniy jadval bilan hisobot bering.	qiyin	15

9	**Latency taqsimoti va ishonch intervali** 5-qadamdagi 6 konfiguratsiyani har biri 5 marta, 1000 takror bilan o'lchang. p50 va p99 uchun bootstrap (1000 qayta tanlash) 95 % ishonch intervalini hisoblang, konfiguratsiyalar orasidagi farq statistik ahamiyatlimi — aniqlang (intervallar kesishadimi). Shuningdek, 450 ta test namunasida 0,2 p.p. aniqlik farqining ishonch intervalini (binomial) baholang va «qancha namuna kerak» savoliga javob bering.	qiyin	15
10	**To'liq Pareto hisoboti o'z modelingiz uchun** O'zingiz tanlagan boshqa arxitektura (masalan, 3 qatlamli kengroq CNN yoki `make_classification` dagi 100 belgili MLP) uchun 1–6 qadamlarning barchasini takrorlang: eksport, ORT darajalari, PyTorch va ORT kvantlash (kamida 2 kalibrlash usuli), yakuniy jadval, Pareto grafigi, energiya bahosi. 4–6 betlik hisobotda aniq ssenariy (masalan, A1 dagi SHV tizimi) uchun konfiguratsiya tanlang va «GPU bo'lsa nima o'zgaradi» bo'limini yozing.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: ko'krak saratoni skrining MLP — ONNX eksport va ORT darajalari oson

Soha: Tibbiyot (onkologiya)

Klinika laboratoriyasi 30 ta biopsiya belgisiga ko'ra xavfli/xavfsiz o'smani tasniflovchi modelni analizator qurilmasidagi CPU'ga joylashtirmoqchi. Qurilma ishlab chiqaruvchisi faqat ONNX formatini qabul qiladi va bitta namuna uchun p99 kechikish 5 ms dan oshmasligini talab qiladi. Sizdan FP32 etalonni o'qitish, ONNX'ga to'g'ri eksport qilish, ORT optimallashtirish darajalarini o'lchash va qaysi daraja qurilma talabiga mos kelishini asoslash so'raladi. Aniqlik birinchi o'rinda: recall 0,95 dan pastga tushmasligi kerak.

Model	MLP 30-64-32-2 (ReLU, Dropout 0,2)
Ma'lumot	sklearn load_breast_cancer (569 namuna, 30 belgi), StandardScaler
Epoxa	15, Adam lr 1e-3, paket 32
Opset	13 va 17 (solishtiriladi)
Batch	1, 8, 64
SLO	p99 <= 5 ms (batch 1), recall >= 0,95
ORT darajalari	DISABLE_ALL, BASIC, EXTENDED, ALL

QADAMLAR

1. Laboratoriya s1 kabi MLP'ni o'qiting (15 epoxa), test to'plamida aniqlik va recall'ni, PyTorch eager batch 1 p50/p95/p99 kechikishini o'lchang.
2. Modelni opset 13 va opset 17 bilan ONNX'ga eksport qiling (dinamik batch o'qi), onnx.checker bilan tekshiring, tugunlar sonini, operatorlar ro'yxatini va fayl hajmini ikkala opset uchun jadvalga yozing.
3. PyTorch va ORT chiqishlarini 64 namunada np.allclose (rtol 1e-3) bilan solishtiring; Dropout eval rejimida grafikdan yo'qolganini ko'rsating.
4. Laboratoriya s2 kabi ORT 4 ta optimallashtirish darajasini batch 1 da o'lchang (takror 500), har daraja uchun p50/p95/p99 va optimallashtirilgan grafikdagi tugunlar sonini jadvalga yozing.
5. Batch 1, 8, 64 uchun throughput va Little qonuni bo'yicha N ni hisoblang; SLO p99 <= 5 ms bajariladimi, qaysi darajada eng katta zaxira qolishini ko'rsating.
6. Xulosa: qurilmaga qaysi opset va daraja tavsiya etiladi, recall saqlanganmi, GPU bo'lsa nima o'zgarardi (2-3 jumla).

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

2-variant. Agro: tuproq sensorlari bo'yicha sug'orish tavsiyasi — chekka qurilma uchun dinamik INT8 oson

Soha: Agro

Fermer xo'jaligi dalalarida 20 ta ko'rsatkichni (namlik, harorat, EC, pH, azot va boshqalar) o'lchaydigan avtonom sensor stansiyalari bor. Stansiya ichidagi kichik ARM protsessor va 2 MB flesh xotira uchun 4 sinfli sug'orish tavsiyasi modeli kerak: model fayli 100 KB dan kichik bo'lishi, batareyada ishlashi kerak. Siz MLP'ni o'qitib, PyTorch dinamik INT8 kvantlashini qo'llaysiz, hajm va aniqlikni solishtirasiz hamda qnnpack dvigatelida (ARM uchun aynan shu dvigatel ishlatiladi) tezlikni o'lchaysiz.

Model	MLP 20-256-128-4
Ma'lumot	make_classification(n_samples=6000, n_features=20, n_informative=12, n_classes=4, random_state=7)
Epoxa	8, Adam lr 2e-3
Hajm cheklovi	<= 100 KB (state_dict)
Batch	1 va 128
Kvantlash dvigateli	qnnpack (ARM muqobili)
Kutilgan aniqlik	>= 85 %

QADAMLAR

1. Sintetik tuproq ma'lumotini yarating (random_state=7), 75/25 ga bo'ling, MLP'ni 8 epoxa o'qiting va FP32 aniqligini yozing.
2. Laboratoriya s3 kabi quantize_dynamic({nn.Linear}, qint8) qo'llang; kvantlangan model tuzilmasini chop eting va DynamicQuantizedLinear qatlamlar sonini ayting.
3. Birinchi qatlam vazni uchun simmetrik kvantlash formulasini qo'lda tekshiring (s = max|w|/127): qayta tiklash xatosi (max, o'rtacha), noyob qiymatlar soni.
4. FP32 va INT8 uchun hajm (KB), aniqlik, p50 kechikish batch 1 va batch 128 ni jadvalga yozing; hajm 100 KB chegarasiga sig'adimi, tekshiring.
5. Qatlam kengligini 256 dan 512 ga oshirib takrorlang: hajm nisbati 4x ga yaqinlashadimi, aniqlik o'sadimi, chegara buziladimi.
6. Xulosa: stansiyaga qaysi konfiguratsiya joylanadi, VNNI yo'q CPU'da tezlik nega oshmagani, ARM NEON'da nimani kutish mumkin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

3-variant. Transport: yo'l belgilarini tanish CNN — statik INT8 va uch kalibrlash usuli o'rta

Soha: Transport

Avtomobil ichidagi yordamchi tizim kameradan kesib olingan kichik belgilar tasvirini tasniflaydi; ishlab chiqaruvchi bort kompyuterida faqat INT8 inferensga ruxsat beradi (issiqlik va quvvat byudjeti tufayli). Siz digits to'plamini belgilar surrogati sifatida ishlatib, kengroq CNN o'qitasiz, ONNX Runtime'da statik kvantlashni MinMax, Percentile va Entropy usullari bilan bajarasiz va noto'g'ri kalibrlash (boshqa yoritilganlik — pikselga doimiy siljish) qanchalik zarar qilishini ko'rsatasiz. Talab: p95 kechikish 2 ms dan kam, aniqlik yo'qotish 0,5 p.p. dan oshmasin.

Model	CNN Conv(1->16)-Conv(16->32)-FC 128-10
Ma'lumot	sklearn digits 8x8, 70/30 bo'linish, random_state=3
Kalibrlash to'plami	256 namuna
Kalibrlash usullari	MinMax, Percentile, Entropy + noto'g'ri (pikselga +0,5 siljish)
SLO	p95 <= 2 ms (batch 1), aniqlik farqi <= 0,5 p.p.
Format	QDQ, per-channel, QUInt8 faollashuv / QInt8 vazn

QADAMLAR

1. Laboratoriya s1/s4 asosida CNN'ni 4 epoxa o'qiting, ONNX'ga eksport qiling, quant_pre_process dan o'tkazing; FP32 aniqligi va p50/p95 ni yozing.
2. 256 namunali real kalibrlash to'plami bilan quantize_static ni MinMax, Percentile va Entropy usullarida bajaring (QDQ, per-channel).
3. Noto'g'ri kalibrlash: xuddi shu 256 namunaga +0,5 doimiy siljish qo'shib (yoritilganlik o'zgarishi) MinMax bilan kvantlang.
4. Har 5 konfiguratsiya (FP32 + 4 INT8) uchun hajm, aniqlik, p50/p95 batch 1, throughput batch 256 va Q/DQ tugunlar sonini jadvalga yig'ing.
5. Kalibrlash to'plami hajmini 32 va 1000 ga o'zgartirib Percentile uchun aniqlikni qayta o'lchang; aniqlik-hajm grafigini chizing.
6. Xulosa: qaysi usul SLO ga mos, noto'g'ri kalibrlash qancha p.p. yo'qotdi, bort kompyuterida (VNNI/NEON) INT8 tezlanishini qanday tekshirish kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

4-variant. Telekom: tarmoq trafigi anomaliyasi uchun autoenkoder — ONNX va yuqori throughput o'rta

Soha: Telekom

Mobil operatorning yadro tarmog'ida har soniyada 40 000 ta oqim yozuvi (flow record) hosil bo'ladi; ularni 16 belgili autoenkoder bilan tekshirib, rekonstruksiya xatosi katta bo'lganlarini anomaliya deb belgilash kerak. Model GPU'siz 2 yadroli virtual mashinada ishlaydi, shuning uchun throughput asosiy mezon: nechta nusxa kerakligini Little qonuni bilan hisoblaysiz. Siz autoenkoderni sintetik trafikda o'qitasiz, ONNX'ga eksport qilasiz, paket o'lchami va oqimlar sonini o'lchaysiz va dinamik INT8 ta'sirini tekshirasiz.

Model	Autoenkoder 16-32-8-32-16 (MSE)
Ma'lumot	sintetik: normal oqimlar make_blobs(1 markaz, 16 belgi, std 1,0) 20000 ta + 2 % anomaliya (std 4,0)
Anomaliya chegarasi	normal xatolarning 99-foizili
Batch	1, 64, 512, 2048
Oqimlar	intra_op 1 va 2
Talab	40 000 yozuv/s, p99 (batch) <= 50 ms
Metrika	precision/recall anomaliya bo'yicha

QADAMLAR

1. Sintetik trafikni yarating, autoenkoderni faqat normal yozuvlarda 10 epoxa o'qiting; test to'plamida rekonstruksiya xatosi va 99-foizil chegara bo'yicha precision/recall ni hisoblang.
2. ONNX'ga eksport qiling (dinamik batch), onnx.checker, PyTorch bilan np.allclose mosligi; tugunlar soni va hajm.
3. Laboratoriya s2 kabi ORT 4 darajasini batch 1 da o'lchang, keyin batch 1/64/512/2048 uchun p50/p99 va throughput (yozuv/s) jadvalini tuzing.
4. intra_op 1 va 2 oqimni batch 2048 da solishtiring, tezlanish koeffitsientini yozing.
5. ORT quantize_dynamic bilan INT8 variant yarating: precision/recall o'zgaradimi, throughput o'zgaradimi (VNNI yo'q — izohlang).
6. Little qonuni: 40 000 yozuv/s uchun har batch o'lchamida nechta nusxa kerak; p99 <= 50 ms sharti bilan optimal batch'ni tanlang.
7. Xulosa va grafik: batch - throughput, batch - p99; tavsiya etilgan konfiguratsiya.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

5-variant. Energetika: soatlik elektr iste'moli prognozi — regressiya modelida kvantlash xatosi oson

Soha: Energetika

Taqsimot kompaniyasi 24 soatlik oyna va ob-havo belgilaridan keyingi soat iste'molini bashorat qiluvchi modelni 1200 ta transformator podstansiyasidagi kontrollerga joylashtirmoqchi. Kontrollerlar eski x86 CPU'li, xotira 256 MB; har 5 daqiqada bitta prognoz kerak, ya'ni tezlik emas, hajm va aniqlik muhim. Siz MLP regressorni o'qitasiz, ONNX'ga eksport qilib ORT'da tekshirasiz, PyTorch va ORT dinamik kvantlashning MAE/MAPE ga ta'sirini o'lchaysiz va tasniflashdan farqli ravishda regressiyada kvantlash xatosi qanday namoyon bo'lishini tushuntirasiz.

Model	MLP 30-128-64-1 (regressiya, MSE)
Ma'lumot	make_regression(n_samples=8000, n_features=30, n_informative=20, noise=8, random_state=11), nishon MinMax [0,1]
Epoxa	20, Adam lr 1e-3
Metrika	MAE, MAPE, R^2
Batch	1 va 256
Cheklov	hajm <= 60 KB, MAE o'sishi <= 2 %
Opset	17

QADAMLAR

1. Sintetik iste'mol ma'lumotini yarating (random_state=11), nishonni [0,1] ga normallang, MLP'ni 20 epoxa o'qiting; test MAE, MAPE, R^2 ni yozing.
2. ONNX eksport (opset 17, dinamik batch), onnx.checker; ORT va PyTorch chiqishlarini 256 namunada maksimal farq bilan solishtiring.
3. ORT ALL darajasida batch 1 va 256 uchun p50 kechikish va throughput o'lchang (laboratoriya s2 funksiyasi bilan).
4. PyTorch quantize_dynamic va ORT quantize_dynamic bilan ikki INT8 variant yarating; hajm, MAE, MAPE, R^2, p50 ni FP32 bilan bitta jadvalga yig'ing.
5. Kvantlash xatosini tahlil qiling: FP32 va INT8 bashoratlari farqining gistogrammasini chizing, eng katta farq qaysi diapazonda ekanini ayting.
6. Xulosa: kontrollerga qaysi variant mos, hajm chegarasi bajarildimi, regressiyada «aniqlik o'zgarmadi» deyish nega noto'g'ri bo'lishi mumkin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

6-variant. Moliya: kredit skoring — tasodifiy o'rmondan MLP distillyatsiyasi va ONNX qiyin

Soha: Moliya (bank krediti)

Bank kredit skoringini sklearn RandomForest bilan bajaradi, lekin yangi onlayn xizmat har bir so'rovga 1 ms dan kam kechikish talab qiladi va faqat ONNX Runtime'ni qo'llab-quvvatlaydi; sandbox'da skl2onnx yo'q. Siz daraxt ansamblini «o'qituvchi» sifatida olib, uning ehtimolliklarini MLP «o'quvchi»ga distillyatsiya qilasiz, MLP'ni ONNX'ga eksport qilasiz va sklearn predict_proba bilan ORT kechikishini solishtirasiz. Keyin o'quvchini INT8 ga kvantlab, AUC va «tasdiqlangan arizalar ulushi» o'zgarishini tekshirasiz — bank uchun AUC 0,01 dan ko'p pasaymasligi shart.

O'qituvchi	RandomForest 200 daraxt, max_depth 10
O'quvchi	MLP 24-128-64-2 (soft-label KL yo'qotish, T=2)
Ma'lumot	make_classification(n_samples=20000, n_features=24, n_informative=14, weights=[0.8,0.2], random_state=21)
Metrika	AUC, aniqlik, tasdiqlash chegarasi 0,5 da tasdiqlanganlar ulushi
SLO	p99 <= 1 ms (batch 1)
Batch	1, 32, 1024
Kvantlash	ORT dinamik + statik Percentile (512 namuna)

QADAMLAR

1. Sintetik kredit ma'lumotini yarating, RandomForest'ni o'qiting, test AUC va bitta so'rov uchun predict_proba p50/p99 kechikishini (300 takror) o'lchang.
2. O'qituvchining ehtimolliklari bilan MLP'ni distillyatsiya qiling (KL + CE aralash, 15 epoxa); o'quvchi AUC'si o'qituvchidan qancha farq qilishini yozing. Taqqoslash uchun oddiy yorliqlar bilan o'qitilgan MLP ham qo'shing.
3. O'quvchini ONNX'ga eksport qiling (laboratoriya s1), ORT 4 daraja va batch 1/32/1024 uchun kechikish va throughput jadvalini tuzing; sklearn RF bilan tezlik nisbatini hisoblang.
4. ORT dinamik va statik (Percentile, 512 namuna) INT8 kvantlash: hajm, AUC, p50, tasdiqlanganlar ulushi; FP32 bilan farq.
5. Pareto tahlili (p50 - AUC) va grafik; RF, MLP FP32, MLP INT8 nuqtalarini bitta grafikda ko'rsating.
6. Tasdiqlangan arizalar ulushi kvantlashdan keyin o'zgargan bo'lsa, bu nechta mijozga ta'sir qiladi (20 000 ta ariza misolida) — hisoblang va izohlang.
7. Xulosa: distillyatsiya qancha aniqlik yo'qotdi va qancha tezlik berdi, skl2onnx bo'lsa qanday bo'lardi, bank uchun tavsiya.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); O'qituvchi-o'quvchi taqqoslash jadvali (AUC, kechikish, hajm)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

7-variant. Ta'lim: talaba javoblari ketma-ketligi uchun kichik transformer bloki — ONNX, fusion va MatMul kvantlash qiyin

Soha: Ta'lim

Adaptiv test platformasi talabaning oxirgi 16 ta javobi (savol identifikatori va to'g'ri/noto'g'ri) asosida keyingi savolga javob ehtimolini bashorat qiladi. Model — bitta self-attention bloki va ikki Linear qatlam; platformaning inferens serveri GPU'siz, bir vaqtda 1500 ta talaba ishlaydi. Siz transformer blokini sintetik ketma-ketliklarda o'qitasiz, ONNX'ga eksport qilasiz, ORT EXTENDED/ALL darajalarida Attention/LayerNorm birlashuvi (fusion) bo'lgan-bo'lmaganini tugunlar bo'yicha tekshirasiz, MatMul og'ir modelda dinamik INT8 kvantlashni baholaysiz va p99 10 ms SLO uchun nusxalar sonini hisoblaysiz.

Model	Embedding(200, 32) + MultiheadAttention(32, 4 bosh) + LayerNorm + FFN 32-64-32 + Linear 32-1
Ma'lumot	sintetik: 20000 ketma-ketlik, uzunlik 16, savol ID 0..199, javob yorlig'i talaba qobiliyati va savol qiyinligidan logistik model bilan
Epoxa	6, Adam lr 1e-3, BCE
Batch	1, 16, 128
SLO	p99 <= 10 ms, 1500 so'rov/s
Opset	17
Kvantlash	PyTorch quantize_dynamic (Linear), ORT quantize_dynamic (MatMul)

QADAMLAR

1. Sintetik javob ketma-ketliklarini yarating (qobiliyat ~ N(0,1), qiyinlik ~ N(0,1), p = sigmoid(qobiliyat - qiyinlik)), modelni 6 epoxa o'qiting, test AUC ni yozing.
2. ONNX'ga eksport qiling (dinamik batch, opset 17); onnx.checker; operatorlar ro'yxatini chop eting (MatMul, Softmax, LayerNormalization yoki uning qismlari bormi).
3. ORT 4 darajasida optimallashtirilgan grafikni faylga saqlab tugunlarni sanang; EXTENDED/ALL da qaysi birlashgan operatorlar (fused) paydo bo'lganini jadvalga yozing.
4. Batch 1/16/128 uchun p50/p99 va throughput; intra_op 1 va 2 oqim taqqosi.
5. PyTorch va ORT dinamik INT8: hajm, AUC, p50 batch 1, throughput batch 128; embedding qatlami kvantlandimi, tekshiring.
6. Little qonuni: 1500 so'rov/s va p99 bo'yicha nusxalar soni FP32 va INT8 uchun; Pareto grafigi (p50 - AUC).
7. Xulosa: fusion qancha tezlik berdi, MatMul og'ir model uchun dinamik kvantlash tezisi tasdiqlandimi, GPU/VNNI bo'lsa nimalar o'zgaradi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Operatorlar/fusion jadvali (4 daraja)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

8-variant. Sanoat: podshipnik vibratsiyasi bo'yicha nosozlik tasnifi — 1D CNN va sezgir qatlam tahlili o'rta

Soha: Sanoat (mashinasozlik)

Zavod nasos stansiyasidagi 300 ta dvigatelning vibratsiya signallari (256 nuqtalik oynalar) 4 sinfga (soz, ichki halqa, tashqi halqa, sharcha nuqsoni) tasniflanadi. Model sexdagi sanoat kompyuteriga joylanadi, u yerda xotira kam va INT8 ma'qul, lekin nosozlikni o'tkazib yuborish qimmat: har sinf bo'yicha recall 0,9 dan pastga tushmasin. Siz 1D CNN'ni sintetik vibratsiya signallarida o'qitasiz, ONNX statik kvantlashni bajarasiz va nodes_to_exclude bilan qaysi Conv qatlami kvantlashga eng sezgir ekanini aniqlab, aralash (INT8 + 1 FP32 qatlam) variantni taklif qilasiz.

Model	1D CNN: Conv1d(1->8,k7)-Pool-Conv1d(8->16,k5)-Pool-Conv1d(16->32,k3)-GAP-Linear 32-4
Ma'lumot	sintetik vibratsiya: 256 nuqta, asosiy sinusoida 50 Hz + sinfga xos zarba chastotasi (0/107/162/141 Hz) + Gauss shovqin std 0,3; 4000 namuna
Epoxa	8, Adam lr 2e-3
Kalibrlash	MinMax va Percentile, 200 namuna, per-channel
Metrika	aniqlik, sinf bo'yicha recall
Cheklov	recall >= 0,9 har sinfda, hajm minimal
Batch	1 va 128

QADAMLAR

1. Sintetik vibratsiya generatorini yozing (4 sinf, zarba chastotalari), 1D CNN'ni o'qiting, test aniqligi va sinflar bo'yicha recall jadvali.
2. ONNX eksport (dinamik batch), quant_pre_process; FP32 ORT p50 batch 1 va throughput batch 128.
3. Laboratoriya s4 kabi statik kvantlash (MinMax, Percentile, QDQ per-channel) va dinamik kvantlash; hajm, aniqlik, recall, p50, Q/DQ tugunlar soni jadvali.
4. nodes_to_exclude bilan uchta Conv va Gemm tugunini navbatma-navbat FP32 da qoldirib 4 ta aralash variant yarating; har biri uchun aniqlik va eng past recall'ni o'lchang.
5. Eng sezgir qatlamni aniqlang va «INT8 + 1 FP32 qatlam» variantini to'liq INT8 va FP32 bilan Pareto (hajm - eng past recall) grafigida ko'rsating.
6. Xulosa: sanoat kompyuteriga qaysi variant, recall talabi bajarildimi, nega birinchi qatlam ko'pincha sezgir bo'ladi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Sezgir qatlam jadvali (4 aralash variant)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

9-variant. Kiberxavfsizlik: login hodisalari bo'yicha hujum aniqlash — keng MLP va 4x hajm chegarasi oson

Soha: Xavfsizlik

Korporativ SIEM tizimi har soniyada 25 000 ta autentifikatsiya hodisasini (40 belgi: vaqt, geolokatsiya, qurilma, xatolar soni va h.k.) hujum/normal deb tasniflaydi. Tasniflagich SIEM agentining ichida ishlaydi, agent paketi o'lchami qat'iy cheklangan, shuning uchun model hajmi kamida 3,5 marta qisqarishi kerak. Siz keng MLP'ni sintetik hodisalarda o'qitasiz, PyTorch dinamik INT8 kvantlash bilan hajm, aniqlik va tezlikni o'lchaysiz hamda batch 1 da p99 1 ms SLO ni tekshirasiz.

Model	MLP 40-512-512-2
Ma'lumot	make_classification(n_samples=30000, n_features=40, n_informative=25, weights=[0.95,0.05], flip_y=0.01, random_state=5)
Epoxa	5, Adam lr 1e-3, paket 128
Hajm talabi	>= 3,5x qisqarish
SLO	p99 <= 1 ms (batch 1)
Batch	1, 256
Metrika	aniqlik, hujum sinfi uchun recall va F1

QADAMLAR

1. Sintetik hodisalarni yarating (muvozanatsiz, 5 % hujum), MLP'ni o'qiting; aniqlik, recall va F1 (hujum sinfi) ni yozing.
2. Laboratoriya s3 kabi quantize_dynamic qo'llang (qnnpack), kvantlangan tuzilmani chop eting; 2-qatlam (512x512) vazni uchun masshtab s, qayta tiklash xatosi va noyob qiymatlar sonini hisoblang.
3. FP32 va INT8: hajm KB, aniqlik, recall, F1, p50/p99 batch 1 va p50 batch 256 jadvali; hajm nisbati 3,5x dan katta-kichikligini tekshiring.
4. Per-channel masshtab bilan qo'lda kvantlab (har qator uchun alohida s) qayta tiklash xatosini per-tensor bilan solishtiring.
5. ONNX'ga eksport qilib ORT ALL da batch 1 p99 ni o'lchang; SLO 1 ms uchun PyTorch INT8, ORT FP32 qaysi biri mos — jadvalda ko'rsating.
6. Xulosa: agentga qaysi format, recall saqlanganmi, VNNI yo'q CPU'da tezlik natijasi va serverda nimani kutish kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

10-variant. Chakana savdo: mahsulot tavsiyasi uchun embedding + MLP — Gather kvantlash va katta paket o'rta

Soha: Chakana savdo

Onlayn do'kon 5000 ta mahsulot va 20 000 ta foydalanuvchi uchun «sotib oladi/olmaydi» ehtimolini hisoblaydi; tungi qayta hisoblashda 100 million juftlik baholanadi, kunduzi esa bitta foydalanuvchi uchun 200 ta nomzodni 20 ms ichida saralash kerak. Model — ikkita embedding va MLP; eng katta hajm embeddinglarda. Siz modelni sintetik o'zaro ta'sirlar (interactions) ustida o'qitasiz, ONNX'ga eksport qilasiz, katta paketlarda throughput'ni, dinamik va statik kvantlashda embeddinglar (Gather) kvantlanadimi-yo'qmi tekshirasiz va tungi hisob uchun vaqtni baholaysiz.

Model	Embedding(20000, 16) + Embedding(5000, 16) + MLP 32-64-1
Ma'lumot	sintetik: foydalanuvchi va mahsulot yashirin vektorlari N(0,1), yorliq = sigmoid(skalyar ko'paytma + shovqin) > 0,5; 300 000 juftlik
Epoxa	3, Adam lr 3e-3, paket 512
Batch (onlayn)	200 nomzod bitta so'rovda
Batch (tungi)	4096, 16384
SLO	p95 <= 20 ms (200 nomzod), tungi 100 mln juftlik <= 2 soat 1 nusxada
Metrika	AUC

QADAMLAR

1. Sintetik o'zaro ta'sirlarni yarating, modelni 3 epoxa o'qiting, test AUC ni yozing; parametrlar sonini embedding va MLP bo'yicha ajrating.
2. ONNX'ga eksport qiling (ikki kirish: user_id, item_id, dinamik batch), onnx.checker; operatorlar ichida Gather borligini ko'rsating; hajm.
3. ORT ALL: batch 200 uchun p50/p95 (SLO tekshiruvi), batch 4096 va 16384 uchun throughput (juftlik/s); 100 mln juftlik uchun vaqtni hisoblang.
4. ORT quantize_dynamic va quantize_static (MinMax, 1024 juftlik kalibrlash): hajm, AUC, throughput; Gather/embedding vaznlari kvantlanganmi — grafik tugunlarini tekshiring.
5. Agar embedding kvantlanmagan bo'lsa, embedding jadvallarini qo'lda INT8 ga (per-qator masshtab) kvantlab qayta tiklang va AUC ni qayta o'lchang; hajm nisbatini hisoblang.
6. Pareto (hajm - AUC) va (throughput - AUC) grafiklari; onlayn va tungi rejim uchun alohida tavsiya bering.
7. Xulosa: hajm qayerda, qaysi kvantlash foyda berdi, 1 nusxada tungi hisob ulgurmasa nechta nusxa kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

11-variant. Logistika: qo'l skanerida yetkazib berish vaqti prognozi — energiya bahosi va batareya umri o'rta

Soha: Logistika

Kuryerlik kompaniyasi 4000 ta batareyali qo'l skaneriga yetkazib berish muddatini (daqiqa) bashorat qiluvchi modelni o'rnatmoqda; har skaner kuniga 1500 ta prognoz qiladi va bir zaryadda kamida 12 soat ishlashi kerak. Shuning uchun asosiy mezon — inferens boshiga energiya (uJ). Siz MLP regressorni o'qitasiz, ONNX FP32 va INT8 dinamik variantlarini 1 va 2 oqimda, batch 1 va 32 da laboratoriya s6 quvvat modeli bilan baholaysiz, lekin quvvat parametrlarini mobil protsessorga moslaysiz (P_bosh 0,4 Vt, P_to'liq 2,5 Vt) va kunlik energiya sarfini mAh ga aylantirasiz.

Model	MLP 18-96-48-1 (regressiya)
Ma'lumot	make_regression(n_samples=10000, n_features=18, n_informative=12, noise=5, random_state=13); nishon 10..180 daqiqaga masshtablangan
Quvvat modeli	P_bosh 0,4 Vt, P_to'liq 2,5 Vt (2 yadro)
Batareya	3000 mAh, 3,8 V
Batch	1 va 32
Oqimlar	1 va 2
Kunlik yuklama	1500 prognoz/skaner
Metrika	MAE (daqiqa)

QADAMLAR

1. Sintetik ma'lumotni yarating, MLP'ni 15 epoxa o'qiting, test MAE ni daqiqada yozing.
2. ONNX eksport + quant_pre_process + ORT quantize_dynamic; FP32 va INT8 uchun hajm va MAE.
3. Laboratoriya s6 energiya funksiyasini mobil quvvat parametrlari bilan ishlating: FP32/INT8 x 1/2 oqim x batch 1/32 uchun latency, CPU bandlik, P, uJ_tolik, uJ_dinamik, inferens/s/Vt jadvali.
4. Kunlik energiya: 1500 prognoz x uJ_tolik ni J va mAh (3,8 V) ga aylantiring; batareyaning necha foizi inferensga ketishini har konfiguratsiya uchun hisoblang.
5. Ustunli grafik (uJ/inferens, log o'q) va eng tejamkor konfiguratsiyani belgilang; 2 oqim batch 1 da nega energiya ko'payganini tushuntiring.
6. Xulosa: skanerga tavsiya, 12 soat talabi bajariladimi, ARM qurilmada RAPL o'rniga qanday o'lchash mumkin (batareya hisoblagichi).

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Energiya jadvali (8 konfiguratsiya) va kunlik mAh hisobi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

12-variant. Shahar: chorraha trafik xaritasi bo'yicha tiqilinch bashorati — kichik CNN, oqimlar va Little qonuni oson

Soha: Shahar

Shahar transport markazi 600 ta chorrahadan har 2 soniyada 6x6 katakli band-bo'shlik xaritasini oladi va 3 sinfli tiqilinch darajasini bashorat qiladi; markaziy server 2 yadroli virtual mashina, GPU yo'q. Umumiy yuklama 3000 so'rov/s ga yetishi mumkin. Siz kichik 2D CNN'ni sintetik xaritalarda o'qitasiz, ONNX'ga eksport qilasiz, ORT darajalari, paket o'lchami va oqimlar sonini o'lchaysiz hamda Little qonuni bilan nechta nusxa kerakligini p99 bo'yicha hisoblaysiz.

Model	CNN Conv(1->8,3x3)-ReLU-Conv(8->16,3x3)-ReLU-GAP-Linear 16-3
Ma'lumot	sintetik 6x6 xarita: sinf bo'yicha bandlik ehtimoli 0,2/0,5/0,8 + fazoviy klaster; 9000 namuna
Epoxa	5
Batch	1, 32, 128, 512
Oqimlar	intra_op 1 va 2
Talab	3000 so'rov/s, p99 <= 3 ms (batch 1)
ORT darajalari	4 ta

QADAMLAR

1. Sintetik 6x6 trafik xaritalari generatorini yozing (3 sinf), CNN'ni o'qiting, aniqlikni yozing.
2. ONNX eksport (dinamik batch, opset 17), onnx.checker, PyTorch bilan moslik; tugunlar soni (BatchNorm ishlatgan bo'lsangiz birlashganini ko'rsating).
3. Laboratoriya s2 kabi 4 ta ORT darajasini batch 1 da o'lchang (takror 500) va tugunlar sonini sanang; PyTorch eager bilan tezlanish.
4. Batch 1/32/128/512 uchun p50/p99, throughput va little_N jadvali; 1 va 2 oqim batch 512 da taqqos.
5. Little qonuni: 3000 so'rov/s uchun batch 1 (p99 bo'yicha) va eng yaxshi batch uchun nusxalar soni; p99 <= 3 ms sharti qaysi batch'gacha bajariladi.
6. Grafik (batch - throughput, batch - p99) va xulosa: markaziy serverga nechta nusxa, GPU bo'lsa qanday o'zgaradi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

13-variant. Ekologiya: havo sifati indeksi tasnifi — INT8 kvantlash va magnitudali siyraklashtirish (pruning) taqqosi o'rta

Soha: Ekologiya

Shahar ekologiya xizmati 150 ta ko'cha sensor qutisiga havo sifati indeksini 5 sinfga ajratuvchi modelni joylashtirmoqda; qutilar 4G orqali yangilanadi va trafik qimmat, shuning uchun model fayli iloji boricha kichik bo'lishi kerak. Sizdan ikki yo'lni solishtirish so'raladi: INT8 kvantlash va vaznlarni magnituda bo'yicha siyraklashtirish (30/60/90 %), hamda ularning kombinatsiyasi. Siz MLP'ni sintetik sensor ma'lumotida o'qitasiz, torch.nn.utils.prune bilan siyraklashtirib qayta o'qitasiz (fine-tune), ONNX'ga eksport qilasiz va hajm (siqilgan .onnx.gz ham), aniqlik, kechikishni Pareto grafigida taqqoslaysiz.

Model	MLP 13-256-128-5
Ma'lumot	make_classification(n_samples=12000, n_features=13, n_informative=9, n_classes=5, random_state=17)
Siyraklashtirish	L1 unstructured 30 %, 60 %, 90 % + 2 epoxa fine-tune
Kvantlash	PyTorch dinamik INT8, ORT dinamik INT8
Hajm o'lchovi	.onnx va gzip siqilgan .onnx.gz (KB)
Batch	1 va 256
Cheklov	aniqlik yo'qotish <= 1 p.p.

QADAMLAR

1. Sintetik havo sifati ma'lumotini yarating, MLP'ni 10 epoxa o'qiting, FP32 aniqligi, hajm va ORT p50 batch 1.
2. torch.nn.utils.prune.l1_unstructured bilan 30/60/90 % siyraklashtiring (prune.remove bilan niqobni qotiring), har birini 2 epoxa fine-tune qiling; nol vaznlar ulushini tekshiring.
3. Har siyrak modelni ONNX'ga eksport qiling; .onnx hajmi o'zgarmaganini ko'rsating va gzip bilan siqilgan hajmni o'lchang (siyraklik siqilishda namoyon bo'ladi).
4. ORT quantize_dynamic ni FP32 va 60 % siyrak modelga qo'llang; jami 8 konfiguratsiya uchun hajm (.onnx, .gz), aniqlik, p50 batch 1, throughput batch 256 jadvali.
5. Pareto (gz hajm - aniqlik) grafigi; 4G orqali 150 qutiga yangilash trafigini (MB) har konfiguratsiya uchun hisoblang.
6. Xulosa: siyraklashtirish CPU'da tezlik beradimi (nega yo'q), qaysi kombinatsiya 1 p.p. chegarasida eng kichik, strukturaviy siyraklashtirish qachon kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); 8 konfiguratsiyali jadval (hajm .onnx/.gz, aniqlik, tezlik)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

14-variant. Sport: akselerometr bo'yicha harakat turini tanish — 1D CNN + GRU, statik kvantlash per-tensor vs per-channel qiyin

Soha: Sport

Fitnes bilaguzuk ishlab chiqaruvchisi 3 o'qli akselerometrning 2 soniyalik oynasi (64 qadam x 3 kanal) bo'yicha 6 ta harakat turini (yurish, yugurish, zinadan chiqish, o'tirish, turish, velosiped) aniqlaydi. Bilaguzuk protsessori faqat INT8 ni tez bajaradi va rekurrent qatlamlar kvantlashda nozik. Siz 1D CNN + GRU modelini sintetik signallarda o'qitasiz, ONNX'ga eksport qilasiz (GRU operatori), ORT statik kvantlashni per-tensor va per-channel rejimlarida, MinMax va Percentile usullarida bajarasiz, GRU tugunini kvantlashdan chiqarib ko'rasiz va 6 sinf bo'yicha chalkashlik matritsasi orqali qaysi harakatlar INT8 da ko'proq zarar ko'rishini aniqlaysiz.

Model	Conv1d(3->16,k5)-ReLU-Pool-GRU(16->32)-Linear 32-6
Ma'lumot	sintetik: 64 qadam x 3 o'q, har sinf uchun chastota/amplituda/ofset naqshi + shovqin std 0,25; 6000 namuna
Epoxa	10, Adam lr 2e-3
Kalibrlash	MinMax va Percentile, 300 namuna
Rejimlar	per-tensor, per-channel, GRU FP32 da (nodes_to_exclude)
Batch	1 va 64
Cheklov	har sinf recall >= 0,85

QADAMLAR

1. Sintetik akselerometr generatorini yozing (6 sinf), modelni o'qiting, aniqlik va chalkashlik matritsasini chop eting.
2. ONNX eksport (dinamik batch, opset 17), onnx.checker; operatorlar ro'yxatida GRU borligini va ORT bilan moslikni ko'rsating; GRU eksportida yuzaga kelgan muammolarni (bo'lsa) hujjatlang.
3. quant_pre_process va ORT dinamik kvantlash; keyin statik kvantlash 4 variant: per-tensor MinMax, per-channel MinMax, per-channel Percentile, per-channel Percentile + GRU FP32 da.
4. Har variant uchun hajm, aniqlik, sinflar bo'yicha recall, p50 batch 1, throughput batch 64, Q/DQ tugunlar soni jadvali.
5. FP32 va eng yomon INT8 variant chalkashlik matritsalarini yonma-yon chizing; qaysi juft sinflar aralashib ketganini va nima uchun (o'xshash chastota) tushuntiring.
6. Pareto (hajm - eng past recall) grafigi; bilaguzukka tavsiya va «recall >= 0,85» talabi bajarilganligini tekshiring.
7. Xulosa: per-tensor va per-channel farqi qancha, GRU ni FP32 da qoldirish qancha hajm/aniqlik savdosini beradi, QAT qachon kerak.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Chalkashlik matritsalari (FP32 va INT8)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

15-variant. Media: sharhlar kayfiyati tasnifi — so'zlar sumkasi MLP va dinamik kvantlash oson

Soha: Media

Yangiliklar portali kuniga 2 million foydalanuvchi sharhini ijobiy/salbiy/neytral deb belgilaydi; model 1000 o'lchamli so'zlar sumkasi (bag-of-words) vektorini oladi va keng Linear qatlamlardan iborat, ya'ni dinamik kvantlash uchun ideal nomzod. Portal serverlarida GPU yo'q, xizmat 2 yadroli konteynerlarda ishlaydi. Siz sintetik siyrak matn belgilarini yaratasiz, MLP'ni o'qitasiz, PyTorch va ORT dinamik INT8 kvantlashini qo'llab hajm, aniqlik va tezlikni o'lchaysiz hamda kunlik 2 million sharhni qayta ishlash uchun nechta konteyner-soat kerakligini hisoblaysiz.

Model	MLP 1000-512-128-3
Ma'lumot	make_classification(n_samples=20000, n_features=1000, n_informative=60, n_classes=3, random_state=23), belgilar >= 0 qilib siyraklashtirilgan (80 % nol)
Epoxa	4, Adam lr 1e-3, paket 128
Batch	1, 64, 512
Kunlik yuklama	2 000 000 sharh
Kvantlash	PyTorch quantize_dynamic, ORT quantize_dynamic
Metrika	makro F1

QADAMLAR

1. Sintetik so'zlar sumkasi ma'lumotini yarating (manfiy qiymatlarni nolga kesib siyraklashtiring), MLP'ni o'qiting; aniqlik va makro F1.
2. Laboratoriya s3 kabi PyTorch dinamik kvantlash: tuzilma, 1-qatlam (1000x512) masshtab va qayta tiklash xatosi, hajm nisbati (4x ga qanchalik yaqin).
3. ONNX eksport + ORT quantize_dynamic; FP32 PyTorch, INT8 PyTorch, FP32 ORT, INT8 ORT — 4 konfiguratsiya uchun hajm, makro F1, p50 batch 1, throughput batch 64 va 512.
4. Har konfiguratsiya uchun 2 mln sharhni qayta ishlash vaqtini (soat) va 2 yadroli konteyner-soatlarni hisoblang; batch 512 da ishlash qancha tejaydi.
5. Grafik: hajm - throughput va Pareto (p50 - F1).
6. Xulosa: portalga qaysi konfiguratsiya, bu CPU'da INT8 tezlik bermagani sababi, VNNI serverda kutilgan tezlanish.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

16-variant. Tibbiyot: EKG aritmiya detektori — 1D CNN, uch mezonli Pareto va implant hajm chegarasi o'rta

Soha: Tibbiyot (kardiologiya)

Kiyiladigan kardiomonitor 1 soniyalik EKG segmentini (250 nuqta) normal/ekstrasistola/fibrillyatsiya deb tasniflaydi; monitor ichidagi mikrokontroller uchun model 50 KB dan oshmasligi, bitta segment 10 ms ichida qayta ishlanishi va xavfli sinflar uchun recall 0,95 dan kam bo'lmasligi kerak. Siz sintetik EKG segmentlarini yaratib 1D CNN'ni o'qitasiz, ONNX eksport, ORT FP32, dinamik INT8 va statik INT8 (ikki kalibrlash usuli) variantlarini tayyorlaysiz va uch mezonli (hajm, kechikish, recall) Pareto tahlilini o'tkazib, hajmni nuqta kattaligi bilan ko'rsatuvchi grafik chizasiz.

Model	Conv1d(1->8,k9)-Pool4-Conv1d(8->16,k7)-Pool4-Conv1d(16->16,k5)-GAP-Linear 16-3
Ma'lumot	sintetik EKG: 250 nuqta, Gauss QRS + P/T to'lqinlar; ekstrasistola = erta keng QRS, fibrillyatsiya = tartibsiz RR + P yo'q; 6000 segment
Epoxa	8
Cheklovlar	hajm <= 50 KB, p99 <= 10 ms, recall (2 xavfli sinf) >= 0,95
Kalibrlash	MinMax, Percentile, 256 segment
Batch	1 (asosiy), 64
Pareto	3 mezon: hajm, p50, eng past recall

QADAMLAR

1. Sintetik EKG generatorini yozing (3 sinf), modelni o'qiting, aniqlik va sinflar bo'yicha recall.
2. ONNX eksport, onnx.checker, PyTorch bilan moslik; hajm va tugunlar soni.
3. ORT 4 daraja batch 1 (p50/p99), batch 64 throughput; SLO p99 <= 10 ms tekshiruvi.
4. Dinamik va statik (MinMax, Percentile, per-channel) INT8; FP32 bilan birga 4 konfiguratsiya uchun hajm, recall (har sinf), p50, Q/DQ tugunlar jadvali.
5. Uch mezonli Pareto: laboratoriya s5 pareto funksiyasini (hajm, p50, eng past recall) uchligiga kengaytiring; grafikda x = p50, y = recall, nuqta kattaligi = hajm.
6. Hajm 50 KB dan oshsa kanallarni 8/16 dan 4/8 ga kamaytirib qayta o'qiting va jadvalga qo'shing; chegara ichidagi eng yaxshi recall qaysi.
7. Xulosa: monitorga tavsiya, aniqlik yo'qotish qaysi sinfda, kalibrlash to'plamida xavfli sinflar yetarli bo'lmasa nima bo'ladi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Uch mezonli Pareto grafigi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

17-variant. Aviatsiya: dvigatel sensorlari bo'yicha qoldiq resurs (RUL) prognozi — ONNX, kvantlash va energiya o'rta

Soha: Aviatsiya

Texnik xizmat kompaniyasi dvigatelning 21 ta sensor ko'rsatkichidan qoldiq ish sikllari sonini (RUL) bashorat qiladi; model parvoz paytida bort ma'lumot yig'uvchi blokida (passiv sovutish, quvvat byudjeti 3 Vt) har soniyada ishlaydi va parvoz tugagach natija yuboriladi. Talab: RMSE 15 sikldan oshmasin, inferens energiyasi minimal bo'lsin. Siz sintetik degradatsiya ma'lumotini yaratib MLP regressorni o'qitasiz, ONNX FP32 va INT8 variantlarini tayyorlaysiz, laboratoriya s6 quvvat modelini bort bloki parametrlariga moslab uJ/inferens va 10 soatlik parvoz uchun Wh ni hisoblaysiz.

Model	MLP 21-128-64-1 (regressiya, Huber yo'qotish)
Ma'lumot	sintetik: 300 dvigatel x 200 sikl; sensorlar = asosiy qiymat + degradatsiya egri chizig'i (RUL kamaygani sari eksponensial siljish) + shovqin
Metrika	RMSE (sikl), MAE
Quvvat modeli	P_bosh 0,8 Vt, P_to'liq 3,0 Vt
Parvoz	10 soat, 1 inferens/s
Batch	1 va 60 (1 daqiqalik to'plam)
Oqimlar	1 va 2

QADAMLAR

1. Sintetik degradatsiya generatorini yozing (RUL = 200 - sikl), dvigatellar bo'yicha 80/20 bo'ling (bir dvigatel ikkala to'plamga tushmasin), MLP'ni o'qiting, RMSE va MAE.
2. ONNX eksport, moslik, ORT ALL p50 batch 1; ORT quantize_dynamic va quantize_static (MinMax, 500 namuna) INT8 variantlari uchun RMSE va hajm.
3. Laboratoriya s6 energiya funksiyasini bort bloki quvvat parametrlari bilan FP32/INT8 statik x 1/2 oqim x batch 1/60 uchun ishlating; uJ_tolik, uJ_dinamik, inferens/s/Vt jadvali.
4. 10 soatlik parvoz uchun energiya (Wh) va 3 Vt byudjetdagi o'rtacha quvvat ulushini hisoblang: har soniyada bitta inferens (batch 1) va har daqiqada 60 talik to'plam (batch 60) rejimlarini solishtiring.
5. Ustunli grafik (uJ/inferens) va RMSE - energiya nuqtali grafigi; Pareto nuqtalarini belgilang.
6. Xulosa: bort blokiga qaysi rejim va format, batch 60 rejimida natija kechikishi (1 daqiqa) qabul qilinadimi, VNNI/ARM da nimani kutish mumkin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Energiya jadvali va parvoz uchun Wh hisobi

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

18-variant. Fintech: tranzaksiya firibgarligini aniqlash — kvantlashning recall va chegaraga ta'siri, Entropy kalibrlash qiyin

Soha: Fintech (to'lovlar)

To'lov tizimi har soniyada 8000 tranzaksiyani real vaqtda tekshiradi; firibgarlik ulushi 1 % va model natijasi ehtimollik chegarasi orqali «bloklash/o'tkazish» qaroriga aylanadi. Infratuzilma jamoasi INT8 ga o'tishni taklif qilmoqda, lekin risk bo'limi «aniqlik o'zgarmadi» degan hisobotga ishonmaydi: muvozanatsiz ma'lumotda aniqlik 99 % bo'lib qolaveradi, recall esa tushishi mumkin. Siz MLP'ni o'qitasiz, ORT dinamik va statik (MinMax, Percentile, Entropy) kvantlashni bajarasiz, har variant uchun PR-egri, recall@precision 0,9 va optimal chegarani hisoblaysiz, hamda kvantlashdan keyin chegarani qayta kalibrlash kerakligini ko'rsatasiz.

Model	MLP 28-128-64-1 (BCE, pos_weight)
Ma'lumot	make_classification(n_samples=60000, n_features=28, n_informative=16, weights=[0.99,0.01], flip_y=0.002, random_state=29)
Epoxa	6, paket 256
Kalibrlash	MinMax, Percentile, Entropy; 1000 namuna (tasodifiy) va 1000 namuna (50 % firibgarlik bilan boyitilgan)
Metrika	AUC-PR, recall@precision=0,9, F1 optimal chegarada
SLO	8000 tranz/s, p99 <= 5 ms
Batch	1 va 256

QADAMLAR

1. Muvozanatsiz sintetik tranzaksiyalarni yarating, MLP'ni pos_weight bilan o'qiting; test aniqlik (nega yuqori), AUC-PR, recall@precision 0,9, optimal chegara.
2. ONNX eksport, quant_pre_process, ORT dinamik kvantlash; statik kvantlash 6 variant: 3 usul x 2 kalibrlash to'plami (tasodifiy / boyitilgan).
3. Har variant uchun FP32 chegarasida recall/precision, keyin shu variant uchun qayta tanlangan optimal chegarada recall/precision; farqni jadvalga yozing.
4. PR-egri chiziqlarini bitta grafikda chizing (FP32 + 7 INT8); AUC-PR farqlari.
5. Laboratoriya s2 kabi batch 1 p99 va batch 256 throughput; Little qonuni bilan 8000 tranz/s uchun nusxalar soni.
6. Pareto (p50 - recall@precision 0,9) va xulosa: qaysi kalibrlash to'plami yaxshiroq va nega, chegarani qayta kalibrlash qancha recall qaytardi, risk bo'limiga yakuniy tavsiya.
7. Bootstrap (500 qayta tanlash) bilan FP32 va eng yaxshi INT8 recall farqining 95 % ishonch intervalini hisoblang — farq statistik ahamiyatlimi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); PR-egri grafigi va chegara jadvali

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

19-variant. Oziq-ovqat sanoati: konveyerda nuqsonli mahsulotni ajratish — ikki sinfli CNN, ORT va 500 dona/s oson

Soha: Oziq-ovqat sanoati

Qandolat fabrikasi konveyeridan soniyasiga 500 ta mahsulot o'tadi; har birining 8x8 kulrang issiqlik xaritasi bo'yicha «soz/nuqsonli» qarori 4 ms ichida chiqishi kerak, aks holda ajratuvchi mexanizm ulgurmaydi. Siz digits to'plamini ikki sinfga aylantirasiz (masalan 0,6,8,9 raqamlari «nuqsonli» surrogati), kengroq CNN o'qitasiz, ONNX eksport qilasiz, ORT darajalari va oqimlarini o'lchaysiz, paketlash orqali throughput'ni oshirasiz va Little qonuni bilan 500 dona/s uchun nusxalar sonini hisoblaysiz.

Model	CNN Conv(1->32)-BN-ReLU-Pool-Conv(32->64)-BN-ReLU-Pool-FC 256-64-2
Ma'lumot	sklearn digits -> 2 sinf (0,6,8,9 = nuqsonli), stratifikatsiya, random_state=31
Epoxa	4
SLO	p99 <= 4 ms (batch 1), 500 dona/s
Batch	1, 4, 16, 64
Oqimlar	1 va 2
Metrika	aniqlik, nuqsonli sinf recall

QADAMLAR

1. Digits'ni ikki sinfga aylantiring, CNN'ni o'qiting; aniqlik va nuqsonli sinf recall'ini yozing; parametrlar soni.
2. ONNX eksport (dinamik batch, opset 17), onnx.checker, BatchNorm birlashganini tugunlar ro'yxatida ko'rsating; PyTorch bilan moslik.
3. Laboratoriya s2 kabi ORT 4 ta darajasini batch 1 da (takror 500) o'lchang, tugunlar soni; PyTorch eager bilan tezlanish.
4. Batch 1/4/16/64 uchun p50/p99 va throughput; 1 va 2 oqim batch 64 da.
5. Little qonuni: 500 dona/s uchun batch 1 da nechta nusxa; batch 4 da paket to'lish kutishi (4/500 s) qo'shilganda p99 SLO ga sig'adimi — hisoblang.
6. Grafik va xulosa: konveyer kompyuteriga konfiguratsiya tavsiyasi; kanallar 32/64 bo'lgani uchun model lab CNN'idan necha marta sekin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

20-variant. Konchilik: NIR spektr bo'yicha ruda tarkibi regressiyasi — to'liq optimallashtirish zanjiri va ishonch intervallari qiyin

Soha: Konchilik

Boyitish fabrikasida konveyer ustidagi spektrometr har 0,5 soniyada 200 kanalli NIR spektr beradi, model ruda tarkibidagi foydali komponent foizini bashorat qiladi va natija flotatsiya reagentini boshqaradi. Model fabrikadagi sanoat kompyuteriga (CPU, GPU yo'q) joylanadi; talablar: RMSE 0,8 % dan oshmasin, p99 20 ms, 6 ta spektrometr. Siz laboratoriyaning 1-6 qadamlarini to'liq takrorlaysiz: etalon va eksport, ORT darajalari va paket, PyTorch dinamik kvantlash, ORT dinamik va statik kvantlash (3 usul), yakuniy jadval va Pareto, energiya bahosi; qo'shimcha ravishda kechikish o'lchovlariga bootstrap ishonch intervallari qo'shasiz.

Model	MLP 200-256-64-1 (regressiya) va 1D CNN Conv1d(1->8,k11)-Pool-Conv1d(8->16,k7)-GAP-Linear (ikki arxitektura)
Ma'lumot	sintetik NIR: 200 kanal, 4 ta Gauss yutilish cho'qqisi, amplitudasi tarkib foizi bilan chiziqli + bazaviy chiziq siljishi + shovqin; 8000 spektr, tarkib 2..40 %
Metrika	RMSE, MAE (foiz birligida)
SLO	p99 <= 20 ms, 12 spektr/s jami
Kalibrlash	MinMax, Percentile, Entropy; 400 spektr
Quvvat modeli	P_bosh 6 Vt, P_to'liq 20 Vt
Statistika	bootstrap 1000 qayta tanlash, 95 % II (p50, p99)
Batch	1, 6, 64

QADAMLAR

1. Sintetik NIR generatorini yozing, ikkala arxitekturani o'qiting, RMSE/MAE; ikkala modelni ONNX'ga eksport qiling va moslikni tekshiring (s1).
2. Ikkala model uchun ORT 4 daraja batch 1, batch 1/6/64 throughput, 1 va 2 oqim (s2); har o'lchovni 5 marta takrorlab p50/p99 uchun bootstrap 95 % ishonch intervalini hisoblang.
3. MLP uchun PyTorch dinamik kvantlash (s3): formula tekshiruvi, hajm, RMSE, tezlik.
4. Ikkala model uchun ORT dinamik va statik kvantlash 3 usul bilan (s4): hajm, RMSE, p50, Q/DQ tugunlar; kalibrlash to'plamini faqat past tarkibli (2..10 %) spektrlardan olib qayta kvantlang — yuqori tarkibda xato qanday o'sadi.
5. Yakuniy jadval (kamida 10 konfiguratsiya) va Pareto (p50 - RMSE), grafikda ishonch intervallarini xato chiziqlari sifatida ko'rsating (s5).
6. Energiya bahosi (s6) sanoat kompyuteri quvvat parametrlari bilan: FP32/INT8, 1/2 oqim, batch 1/6; 6 spektrometr uchun yillik kVt-soat va narx (0,08 USD/kVt-soat).
7. 4-6 betlik hisobot: fabrika uchun konfiguratsiya tavsiyasi, statistik ahamiyatli farqlar qaysilar, «GPU bo'lsa nima o'zgaradi» bo'limi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan); Yakuniy jadval (>= 10 konfiguratsiya) ishonch intervallari bilan; 4-6 betlik hisobot (PDF)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20